Chapter 9 ▪ Collections

The data structures that you choose can make a big difference when you try to implement methods in a natural style or are concerned with performance. Do you need to search quickly through thousands (or even millions) of sorted items? Do you need to rapidly insert and remove elements in the middle of an ordered sequence? Do you need to establish associations between keys and values?

This chapter shows how the Java Collections Framework can help you accomplish the traditional data structuring needed for serious programming. In college computer science programs, a course called *Data Structures* usually takes a semester to complete, and there are many, many books devoted to this important topic. Our coverage differs from that of a college course; we will skip the theory and just look at how to use the collection classes in the Java API.

9.1. The Java Collections Framework

The initial release of Java supplied only a small set of classes for the most useful data structures: `Vector`, `Stack`, `Hashtable`, `BitSet`, and the `Enumeration` interface that provides an abstract mechanism for visiting elements in an arbitrary container. That was certainly a wise choice—it takes time and skill to come up with a comprehensive collection class library.

With the advent of Java 1.2, the designers felt that the time had come to roll out a full-fledged framework for data structures. They faced a number of conflicting design challenges. They wanted the library to be small and easy to learn. They did not want the complexity of the Standard Template Library (or STL) of C++, but they wanted the benefit of "generic algorithms" that STL pioneered. They wanted the legacy classes to fit into the new framework. As all designers of collections libraries do, they had to make some hard choices, and they came up with a number of idiosyncratic design decisions along the way. In this section, we will explore the basic design of the Java Collections Framework, demonstrate how to put it to work, and explain the reasoning behind some of the more controversial features.

9.1.1. Separating Collection Interfaces and Implementation

As is common with modern data structure libraries, the Java Collections Framework separates *interfaces* and *implementations*. Let us look at that separation with a familiar data structure, the *queue*.

A *queue interface* specifies that you can add elements at the tail end of the queue, remove them at the head, and find out how many elements are in the queue. You

use a queue when you need to collect objects and retrieve them in a “first in, first out” fashion (see **Figure 9.1**).

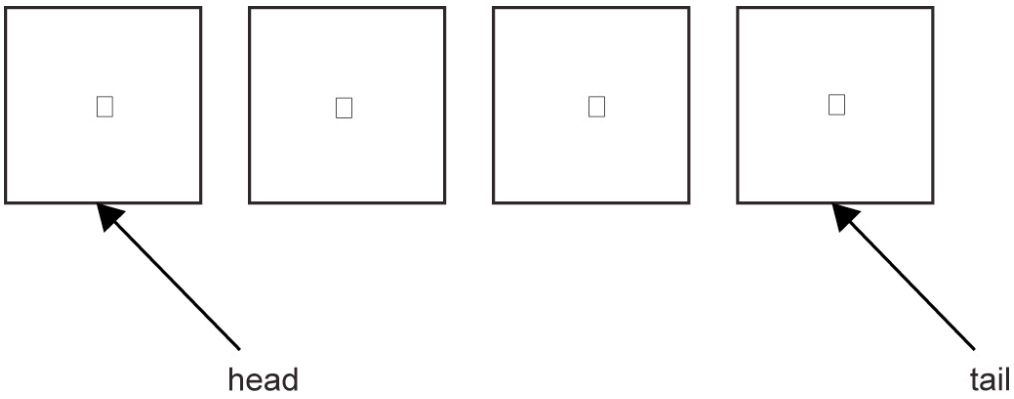


Figure 9.1: A queue

A minimal form of a queue interface might look like this:

```
public interface Queue<E> { // a simplified form of the interface in the Java API
    void add(E e);
    E remove();
    int size();
}
```

The interface tells you nothing about how the queue is implemented. Of the two common implementations of a queue, one uses a “circular array” and one uses a linked list (see **Figure 9.2**).

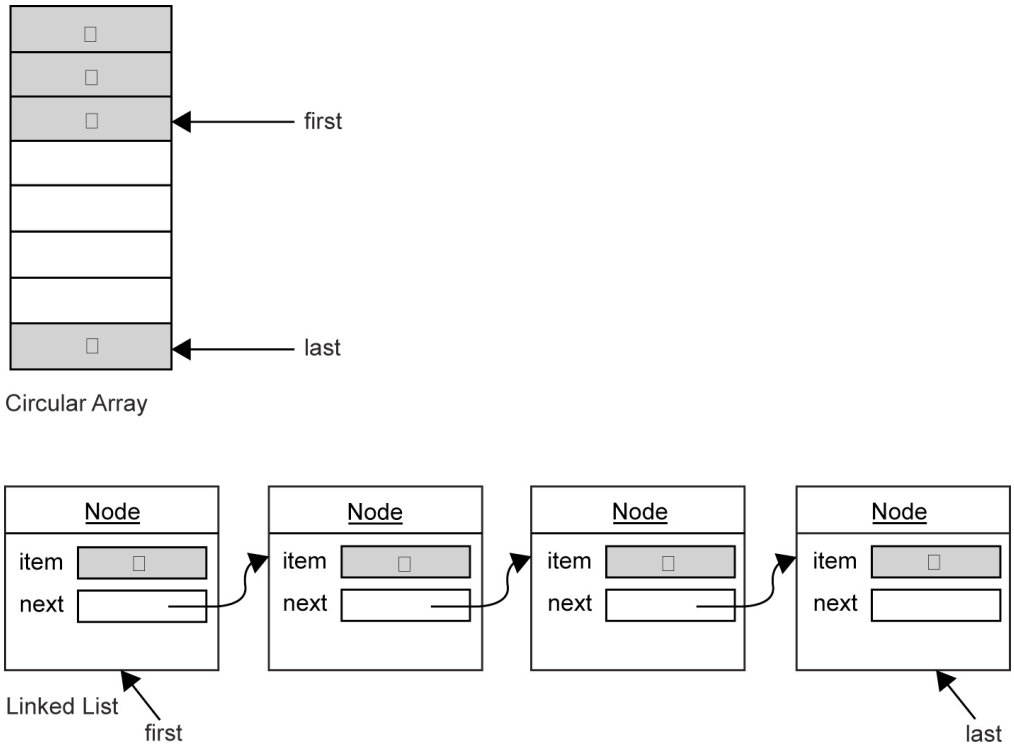


Figure 9.2: Queue implementations

Each implementation can be expressed by a class that implements the `Queue` interface. In the Java API, these classes are called `ArrayDeque` and `LinkedList`.

```
public class ArrayDeque<E> implements Queue<E> { // simplified from the Java API
    private int first;
    private int last;
    private E[] elements;

    public ArrayDeque(int numElements) { . . . }
    public void add(E e) { . . . }
    public E remove() { . . . }
    public int size() { . . . }
    . . .
}

public class LinkedList<E> implements Queue<E> { // simplified from the Java API
    private Node<E> first;
    private Node<E> last;

    public LinkedList() { . . . }
    public void add(E e) { . . . }
    public E remove() { . . . }
    public int size() { . . . }
    . . .
}
```

When you use a queue in your program, you don't need to know which implementation is actually used once the collection has been constructed. Therefore, it makes sense to use the concrete class *only* when you construct the collection object. Use the *interface type* to hold the collection reference.

```
Queue<Customer> expressLane = new ArrayDeque<>(100);
expressLane.add(new Customer("Harry"));
```

With this approach, if you change your mind, you can easily use a different implementation. You only need to change your program in one place—in the constructor call. If you decide that a linked list is a better choice after all, your code becomes

```
Queue<Customer> expressLane = new LinkedList<>();
expressLane.add(new Customer("Harry"));
```

There is another reason to stick with the interface. Both the `ArrayDeque` and `LinkedList` classes have methods that are unrelated to queueing. Accidentally using one of them will be reported as a compile-time error, since you can't invoke them on a variable of type `Queue`.

Why would you choose one implementation over another? The interface says nothing about the efficiency of an implementation. A circular array is more memory-efficient. However, the circular array has a finite capacity and needs to be reallocated whenever it fills up. In fact, measurements show that an `ArrayDeque`

generally outperforms a `LinkedList` because it has less impact on the garbage collector. When the `ArrayDeque` appeared in Java 6, the programmers who made use of the `Queue` interface were able to swap out the `LinkedList` by changing a single line of code.

9.1.2. The `Collection` Interface

The fundamental interface for collection classes in the Java Collections Framework is the `Collection` interface. The interface has two fundamental methods:

```
public interface Collection<E> {
    boolean add(E element);
    Iterator<E> iterator();
    . . .
}
```

There are several methods in addition to these two; we will discuss them later.

The `add` method adds an element to the collection. The `add` method returns `true` if adding the element actually changes the collection, and `false` if the collection is unchanged. For example, if you try to add an object to a set and the object is already present, the `add` request has no effect because sets reject duplicates.

The `iterator` method returns an object that implements the `Iterator` interface. With that object, you can visit the elements in the collection one by one. We discuss iterators in the next section.

9.1.3. Iterators

The `Iterator` interface has four methods:

```
public interface Iterator<E> {
    boolean hasNext();
    E next();
    default void remove();
    default void forEachRemaining(Consumer<? super E> action);
}
```

By repeatedly calling the `next` method, you can visit the elements from the collection one by one. However, if you reach the end of the collection, the `next` method throws a `NoSuchElementException`. Therefore, you need to call the `hasNext` method before calling `next`. That method returns `true` if the iterator has not yet reached the end, and there is at least one more element to visit. If you want to inspect all elements in a collection, request an iterator and then keep calling the `next` method while `hasNext` returns `true`. For example:

```
Collection<String> coll = . . .;
Iterator<String> iter = coll.iterator();
while (iter.hasNext()) {
    String element = iter.next();
    do something with element
}
```

You can write such a loop more concisely as the “for each” loop:

```
for (String element : coll) {
    do something with element
}
```

The compiler simply translates the “for each” loop into a loop with an iterator.

The “for each” loop works with any object that implements the `Iterable` interface, an interface with an abstract method:

```
public interface Iterable<E> {
    Iterator<E> iterator();
    . . .
}
```

The `Collection` interface extends the `Iterable` interface. Therefore, you can use the “for each” loop with any collection in the Java Collections Framework.

Instead of writing a loop, you can call the `Collection.forEach` or `Iterator.forEachRemaining` methods with a lambda expression that consumes an element. The lambda expression is invoked on all elements of the collection, or the remaining elements that the iterator can visit.

```
coll.forEach(element -> do something with element);
iter.forEachRemaining(element -> do something with element);
```

The order in which the elements are visited depends on the collection type. If you iterate over an `ArrayList`, the iterator starts at index `0` and increments the index in each step. However, if you visit the elements in a `HashSet`, you will get them in an essentially random order. You can be assured that you will encounter all elements of the collection during the course of the iteration, but you cannot make any assumptions about their ordering. This is usually not a problem because the ordering does not matter for computations such as computing totals or counting matches.



Note: Old-timers will notice that the `next` and `hasNext` methods of the `Iterator` interface serve the same purpose as the `nextElement` and

`hasMoreElements` methods of an `Enumeration`. The designers of the Java Collections Framework could have chosen to make use of the `Enumeration` interface. But they disliked the cumbersome method names and instead introduced a new interface with shorter method names.

There is an important conceptual difference between iterators in the Java collections library and iterators in other libraries. In traditional collections libraries, such as the Standard Template Library of C++, iterators are modeled after array indexes. Given such an iterator, you can look up the element that is stored at that position, much like you can look up an array element `a[i]` if you have an array index `i`. Independently of the lookup, you can advance the iterator to the next position. This is the same operation as advancing an array index by calling `i++`, without performing a lookup. However, the Java iterators do not work like that. The lookup and position change are tightly coupled. The only way to look up an element is to call `next`, and that lookup advances the position.

Instead, think of Java iterators as being *between elements*. When you call `next`, the iterator *jumps over* the next element, and it returns a reference to the element that it just passed (see **Figure 9.3**).

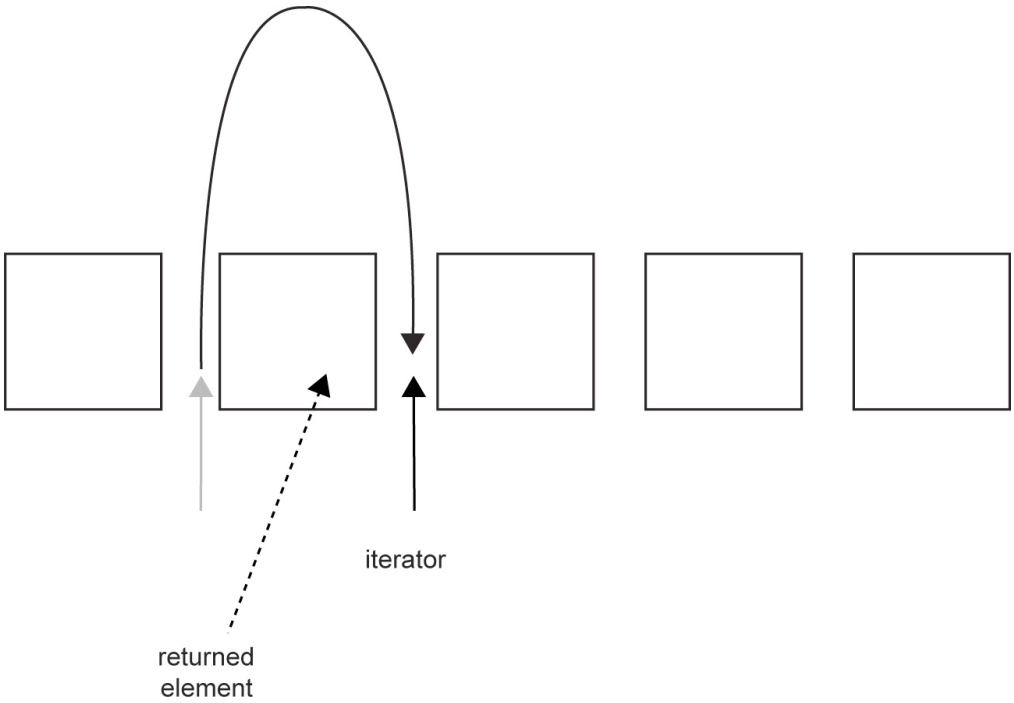


Figure 9.3: Advancing an iterator



Note: Here is another useful analogy. You can think of `Iterator.next` as the equivalent of `InputStream.read`. Reading a byte from a stream automatically "consumes" the byte. The next call to `read` consumes and returns the next byte from the input. Similarly, repeated calls to `next` let you read all elements in a collection.

The `remove` method of the `Iterator` interface removes the element that was returned by the last call to `next`. In many situations, that makes sense—you need to see the element before you can decide that it is the one that should be removed.

But if you want to remove an element in a particular position, you still need to skip past the element. For example, here is how you remove the first element in a collection of strings:

```
Iterator<String> iter = coll.iterator();
iter.next(); // skip over the first element
iter.remove(); // now remove it
```

More importantly, there is a dependency between the calls to the `next` and `remove` methods. It is illegal to call `remove` if it wasn't preceded by a call to `next`. If you try, an `IllegalStateException` is thrown.

If you want to remove two adjacent elements, you cannot simply call

```
iter.remove();
iter.remove(); // ERROR
```

Instead, you must first call `next` to jump over the element to be removed.

```
iter.remove();
iter.next();
iter.remove(); // OK
```

9.1.4. Generic Utility Methods

The `Collection` and `Iterator` interfaces are generic, which means you can write utility methods that operate on any kind of collection. For example, here is a generic method that tests whether an arbitrary collection contains a given element:

```
public static <E> boolean contains(Collection<E> c, Object obj) {
    for (E element : c)
        if (element.equals(obj))
            return true;
    return false;
}
```

The designers of the Java Collections Framework decided that some of these utility methods are so useful that the framework should make them available. That way, API users don't have to keep reinventing the wheel. The `contains` method is one such method.

In fact, the `Collection<E>` interface declares quite a few useful methods that all implementing classes must supply:

```
int size()
boolean isEmpty()
boolean contains(Object obj)
boolean containsAll(Collection<?> c)
boolean add(E obj)
boolean addAll(Collection<? extends E> from)
boolean remove(Object obj)
boolean removeAll(Collection<?> c)
boolean retainAll(Collection<?> c)
void clear()
Object[] toArray()
T[] toArray(T[] a)
```



Note: The `Collection` interface declares the `equals` and `hashCode` methods that are inherited from `Object`. There is no technical need to include these declarations. It merely provides a good place for adding collection-specific API documentation. The `Set` and `List` subinterfaces mandate specific behavior for `equals`. Other collections can inherit `Object.equals`, or override it and compare the elements in some way. In the latter case, the `hashCode` method also needs to be overridden.

Many of these methods are self-explanatory; you will find full documentation in the API notes at the end of this section.



Caution: The Java Collections Framework was designed before generic types were added to Java. For backwards compatibility, the `contains` and `remove` methods have a parameter of type `Object` and not `E`. The `containsAll`, `removeAll`, and `retainAll` methods have a parameter of type `Collection<?>` and not `Collection<? extends E>`.

This means that type errors may not be detected at compile time. As an example, consider this code snippet where accidentally a `String` is removed from a collection of `Path` objects:

```
Collection<Path> paths = . . .;
paths.remove("/tmp"); // Compiles, but can have no effect
```

Of course, it is a bother if every class that implements the `Collection` interface has to supply so many routine methods. To make life easier for implementors, the framework supplies a class `AbstractCollection` that leaves the fundamental methods `size` and `iterator` abstract but implements the routine methods in terms of them. For example:

```
public abstract class AbstractCollection<E> implements Collection<E> {
    . . .
    public abstract Iterator<E> iterator();
    public abstract int size();
```

```
    public boolean isEmpty() {
        return size() == 0;
    }

    public boolean contains(Object obj) {
        Iterator<E> it = iterator();
        if (o == null) {
            while (it.hasNext()) if (it.next() == null) return true;
        }
        else {
            while (it.hasNext()) if (o.equals(it.next())) return true;
        }
        return false;
    }
    . . .
}
```

A concrete collection class can now extend the `AbstractCollection` class. It is up to the concrete collection class to supply `iterator` and `size` methods. Mutable collections also need an `add` method. The other methods have been taken care of by the `AbstractCollection` superclass. However, if the subclass has a more efficient way of implementing `contains`, it is free to do so.

This approach is a bit outdated. It would be nicer if the methods were default methods of the `Collection` interface, but default methods didn't exist when the Java Collections Framework was designed. However, several default methods have been added. Three of them deal with streams (discussed in Volume II). See **Section 9.6.6** for the `toArray` method. In addition, there is a useful method

```
default boolean removeIf(Predicate<? super E> filter)
```

for removing elements that fulfill a condition.

java.util.Collection<E> **1.2**

- `Iterator<E> iterator()`
  returns an iterator that can be used to visit the elements in the collection.
- `int size()`
  returns the number of elements currently stored in the collection.
- `boolean isEmpty()`
  returns `true` if this collection contains no elements.
- `boolean contains(Object obj)`
  returns `true` if this collection contains an object equal to `obj`.
- `boolean containsAll(Collection<?> other)`
  returns `true` if this collection contains all elements in the other collection.

- `boolean add(E element)`
  attempts to add an element to the collection. Returns `true` if the collection changed as a result of this call.
- `boolean addAll(Collection<? extends E> other)`
  adds all elements from the other collection to this collection. Returns `true` if the collection changed as a result of this call.
- `boolean remove(Object obj)`
  attempts to remove an object equal to `obj` from this collection. Returns `true` if a matching object was removed.
- `boolean removeAll(Collection<?> other)`
  removes from this collection all elements from the other collection. Returns `true` if the collection changed as a result of this call.
- `default boolean removeIf(Predicate<? super E> filter)` 8
  removes all elements for which `filter` returns `true`. Returns `true` if the collection changed as a result of this call.
- `void clear()`
  removes all elements from this collection.
- `boolean retainAll(Collection<?> other)`
  removes all elements from this collection that do not equal one of the elements in the other collection. Returns `true` if the collection changed as a result of this call.
- `Object[] toArray()`
  returns an array of the objects in the collection.
- `<T> T[] toArray(IntFunction<T[]> generator)` 11
  returns an array of the objects in the collection. The array is constructed with the `generator`, which is typically a constructor expression `T[]::new`.

java.util.Iterator<E> 1.2

- `boolean hasNext()`
  returns `true` if there is another element to visit.
- `E next()`
  returns the next object to visit. Throws a `NoSuchElementException` if the end of the collection has been reached.
- `void remove()`
  removes the last visited object. This method must immediately follow an element visit. If the collection has been modified since the last element visit, this method throws an `IllegalStateException`. Iterators are not required to support this operation. The default implementation throws an `UnsupportedOperationException`.
- `default void forEachRemaining(Consumer<? super E> action)` 8
  visits elements and passes them to the given action until no elements remain or the action throws an exception.

9.2. Interfaces in the Collections Framework

The Java Collections Framework defines a number of interfaces for different types of collections. These are shown in **Figure 9.4**, except for interfaces for concurrent programming, which are covered in **Chapter 10**.

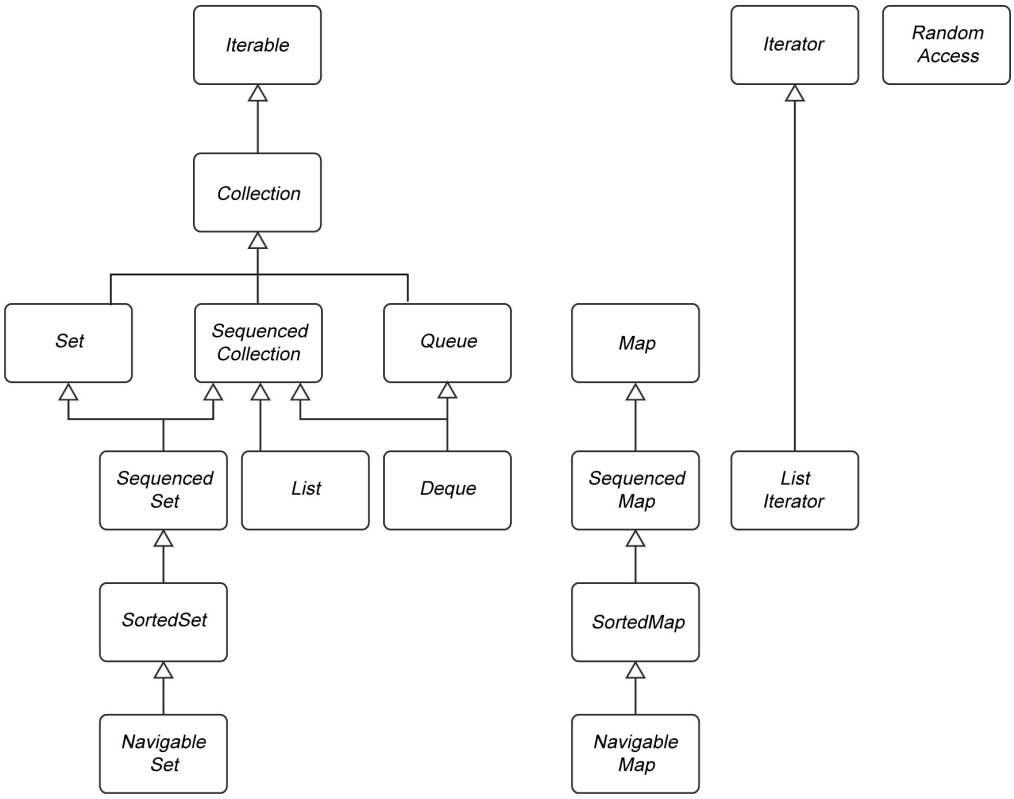


Figure 9.4: The interfaces of the Java Collections Framework

There are two fundamental interfaces for collections: `Collection` and `Map`. As you already saw, you insert elements into a collection with a method

```
boolean add(E element)
```

However, maps hold key/value pairs, and you use the `put` method to insert them:

```
V put(K key, V value)
```

To read elements from a collection, visit them with an iterator. However, you can read values from a map with the `get` method:

```
V get(Object key)
```

A `List` is an *ordered collection*. Elements are added into a particular position in the container. An element can be accessed in two ways: by an iterator or by an integer index. The latter is called *random access* because elements can be visited in any order. In contrast, when using an iterator, one must visit them sequentially.

The `List` interface defines several methods for random access:

```
void add(int index, E element)
E remove(int index)
E get(int index)
E set(int index, E element)
```



Caution: With a `List<Integer>`, there are two `remove` methods:

```
boolean remove(int index) // Removes the element with the given index
boolean remove(Integer o) // Removes the element equal to o
```

When calling `remove` with a parameter of type `int`, the first method is chosen. No boxing is considered, and there is no warning. This may not be what you want:

```
List<Integer> ids = . . .;
int id = . . .;
if (id == 0) ids.remove(id); // Removes index 0, not the element with value 0
```



Note: The API documentation for the `List` interface defines the behavior of the `equals` method for lists. For a list to be equal to another object, the other object must also be a list of some kind. Both lists must have the same size. When iterating over each list, corresponding elements must be equal.

Frankly, there is a problem with the `List` interface in the Java Collections Framework. There are two kinds of ordered collections, with very different performance tradeoffs. An ordered collection that is backed by an array has fast random access, and it makes sense to use the `List` methods with an integer index. In contrast, a linked list, while also ordered, has slow random access, and it is best traversed sequentially with an iterator. It would have been better to provide two interfaces.



Note: To avoid carrying out random access operations for linked lists, Java 1.4 introduced a tagging interface, `RandomAccess`. That interface has no methods, but you can use it to test whether a particular collection supports efficient random access:

```
if (c instanceof RandomAccess) {
    random traversal
}
else {
    sequential traversal
}
```

The `Set` interface has the same instance methods as the `Collection` interface, but the behavior of the methods is more tightly defined. The `add` method of a set should reject duplicates. The `equals` method of a set should be defined so that two sets are identical if they have the same elements, but not necessarily in the same order. The `hashCode` method should be defined so that two sets with the same elements yield the same hash code.

Why make a separate interface if the method signatures are the same? Conceptually, not all collections are sets. Making a `Set` interface enables programmers to write methods that accept only sets.

To see why this distinction matters, consider equality testing. Two lists are equal if they have the same elements in the same order. Two sets are equal if they have the same elements in some order. The `equals` method for a class implementing the `Set` interface needs to ensure that the argument is also a `Set` and not just any `Collection`.

The `SortedSet` and `SortedMap` interfaces expose the comparator object used for sorting, and they define methods to obtain views of subsets of the collections. We discuss these in **Section 9.5.3**.

The interfaces `NavigableSet` and `NavigableMap` contain additional methods for finding the next or previous element in sorted sets and maps. The `TreeSet` and `TreeMap` classes implement these interfaces. The navigation operations can be efficiently implemented in tree-based data structures.

Java 21 introduces the `SequencedCollection<E>` interface that provides uniform access to the first and last elements of a collection and reverse traversal.

```
E getFirst()
E getLast()
void addFirst(E e)
void addLast(E e)
E removeFirst()
E removeLast()
SequencedCollection<E> reversed()
```

Previously, these operations were carried out by different methods in lists, sets, and deques. The `SequencedSet` subinterface sharpens the return type of the `reversed` method to `SequencedSet`. The `SequencedMap` interface has analogous methods for maps.

9.3. Concrete Collections

Table 9.1 shows the collections in the Java Collections Framework and briefly describes the purpose of each collection class. (For simplicity, I omit the thread-safe collections that will be discussed in **Chapter 10**.)

All classes in **Table 9.1** implement the `Collection` interface, with the exception of the classes with names ending in `Map`. Those classes implement the `Map` interface instead. We will discuss maps in **Section 9.4**.

Table 9.1: Concrete Collections in the Java Collections Framework

Collection Type	Description	See
`ArrayList`	An indexed sequence that grows and shrinks dynamically	**Section 9.3.2**
`LinkedList`	An ordered sequence that allows efficient insertion and removal at any location	**Section 9.3.1**
`ArrayDeque`	A double-ended queue that is implemented as a circular array	**Section 9.3.5**
`HashSet`	An unordered collection that rejects duplicates	**Section 9.3.3**
`TreeSet`	A sorted set	**Section 9.4.5**
`EnumSet`	A set of enumerated type values	**Section 9.4.6**
`LinkedHashSet`	A set that remembers the order in which elements were inserted	**Section 9.4.5**
`PriorityQueue`	A collection that allows efficient removal of the smallest element	**Section 9.3.6**
`HashMap`	A data structure that stores key/value associations	**Section 9.4.1**
`TreeMap`	A map in which the keys are sorted	**Section 9.4.1**
`EnumMap`	A map in which the keys belong to an enumerated type	**Section 9.4.6**
`LinkedHashMap`	A map that remembers the order in which entries were added	**Section 9.4.5**
`WeakHashMap`	A map with values that can be reclaimed by the garbage collector if they are not used elsewhere	**Section 9.4.4**

Collection Type	Description	See
`IdentityHashMap`	A map with keys that are compared by `==`, not `equals`	Section 9.4.7

Figure 9.5 shows the relationships between these classes.

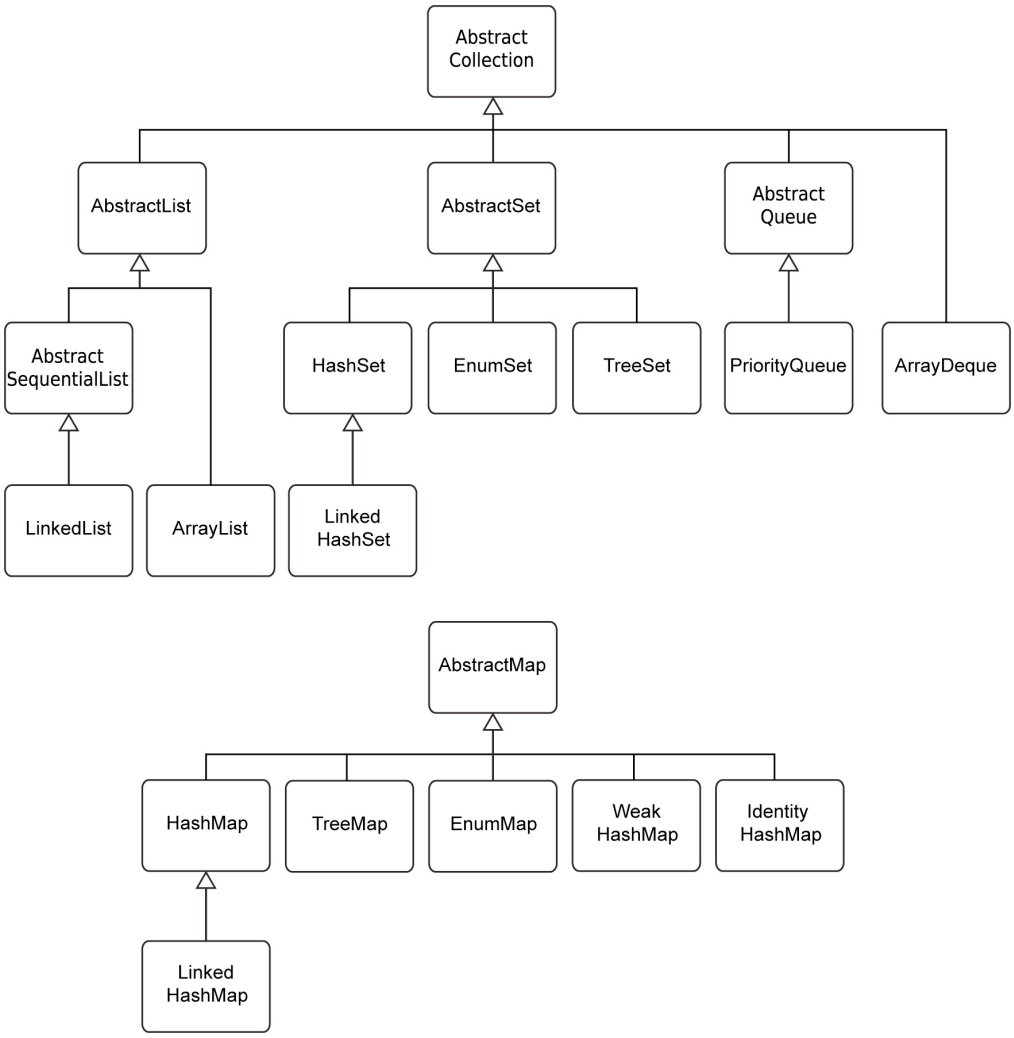


Figure 9.5: Classes in the Java Collections Framework

9.3.1. Linked Lists

We already used arrays and their dynamic cousin, the `ArrayList` class, for many examples in this book. However, arrays and array lists suffer from a major drawback. Removing an element from the middle of an array is expensive since all array elements beyond the removed one must be moved toward the beginning of the array (see Figure 9.6). The same is true for inserting elements in the middle.

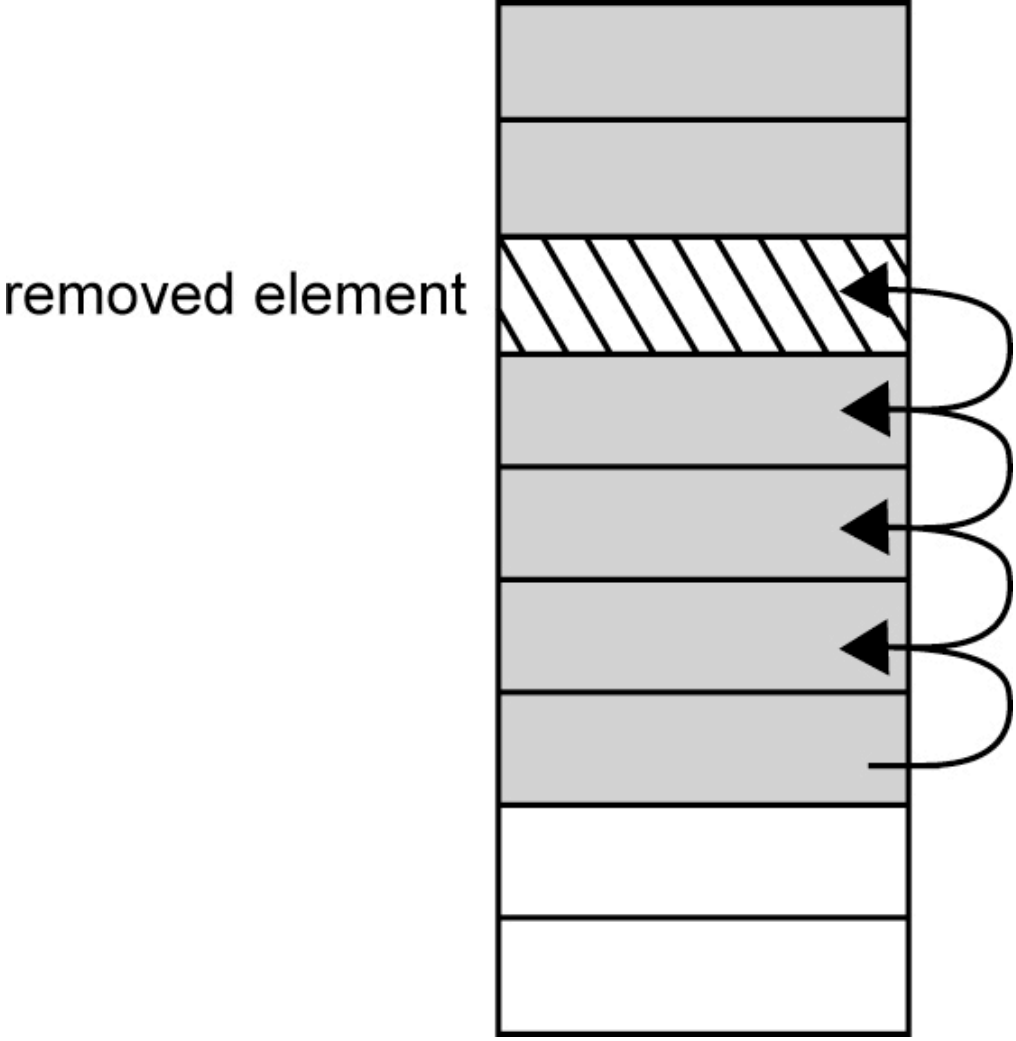


Figure 9.6: Removing an element from an array

Another well-known data structure, the *linked list*, solves this problem. Where an array stores object references in consecutive memory locations, a linked list stores each object in a separate *link*. Each link also stores a reference to the next link in the sequence. In the Java Collections Framework, all linked lists are actually *doubly linked*; that is, each link also stores a reference to its predecessor (see **Figure 9.7**).

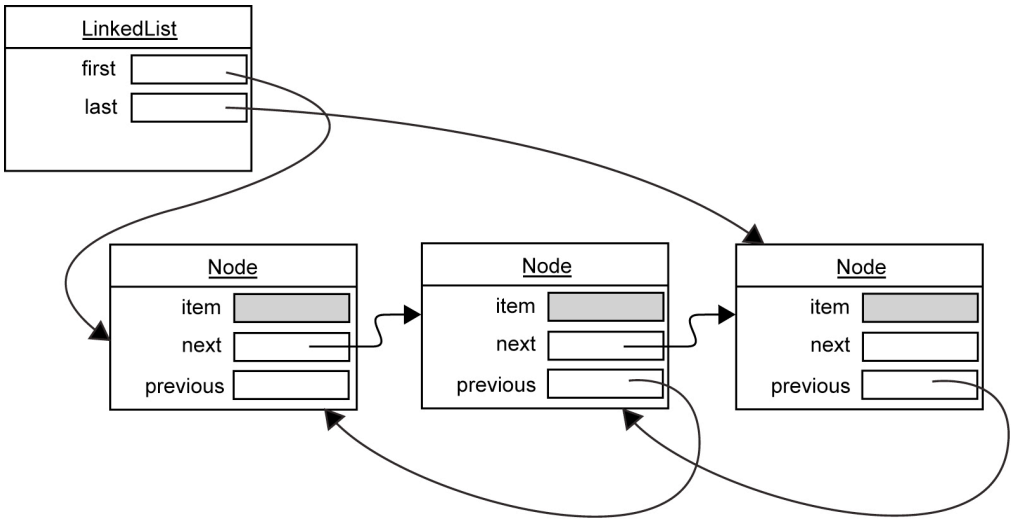


Figure 9.7: A doubly linked list

Removing an element from the middle of a linked list is an inexpensive operation—only the links around the element to be removed need to be updated (see Figure 9.8).

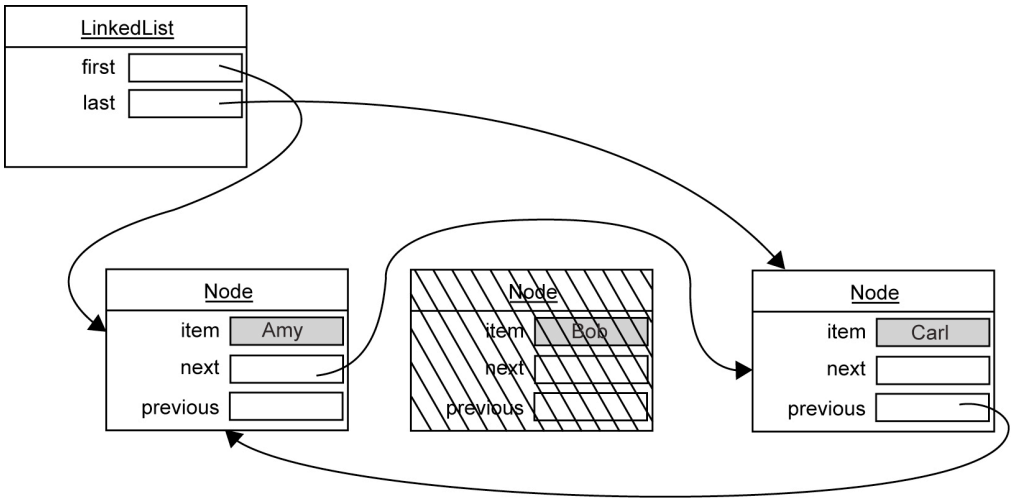


Figure 9.8: Removing an element from a linked list

Perhaps you once took a data structures course in which you learned how to implement linked lists. You may have bad memories of tangling up the links when removing or adding elements in the linked list. If so, you will be pleased to learn that the Java Collections Framework supplies a class `LinkedList` ready for you to use.

The following code example adds three elements and then removes the second one:

```
var staff = new LinkedList<String>();
staff.add("Amy");
staff.add("Bob");
staff.add("Carl");
Iterator<String> iter = staff.iterator();
String first = iter.next(); // visit first element (Amy)
String second = iter.next(); // visit second element (Bob)
iter.remove(); // remove last visited element (Bob)
```

There is, however, an important difference between linked lists and generic collections. A linked list is an *ordered collection* in which the position of the objects matters. The `LinkedList.add` method adds the object to the end of the list. But you will often want to add objects somewhere in the middle of a list. This position-dependent `add` method is the responsibility of an iterator, since iterators describe positions in collections. Using iterators to add elements makes sense only for collections that have a natural ordering. For example, the *set* data type that we discuss in the next section does not impose any ordering on its elements. Therefore, there is no `add` method in the `Iterator` interface. Instead, the Java Collections Framework supplies a subinterface `ListIterator` that contains an `add` method:

```
interface ListIterator<E> extends Iterator<E> {
    void add(E element);
    . . .
}
```

Unlike `Collection.add`, this method does not return a `boolean`—it is assumed that the `add` operation always modifies the list.

In addition, the `ListIterator` interface has two methods that you can use for traversing a list backwards.

```
boolean hasPrevious()
E previous()
```

Like the `next` method, the `previous` method returns the object that it skipped over.

The `listIterator` method of the `LinkedList` class returns an iterator object that implements the `ListIterator` interface.

```
ListIterator<String> iter = staff.listIterator();
```

The `add` method adds the new element *before* the iterator position. For example, the following code skips over the first element in the linked list and adds `"Juliet"` before the second element (see **Figure 9.9**):

```
var staff = new LinkedList<String>();
staff.add("Amy");
staff.add("Bob");
staff.add("Carl");
ListIterator<String> iter = staff.listIterator();
iter.next(); // skip past first element
iter.add("Juliet");
```

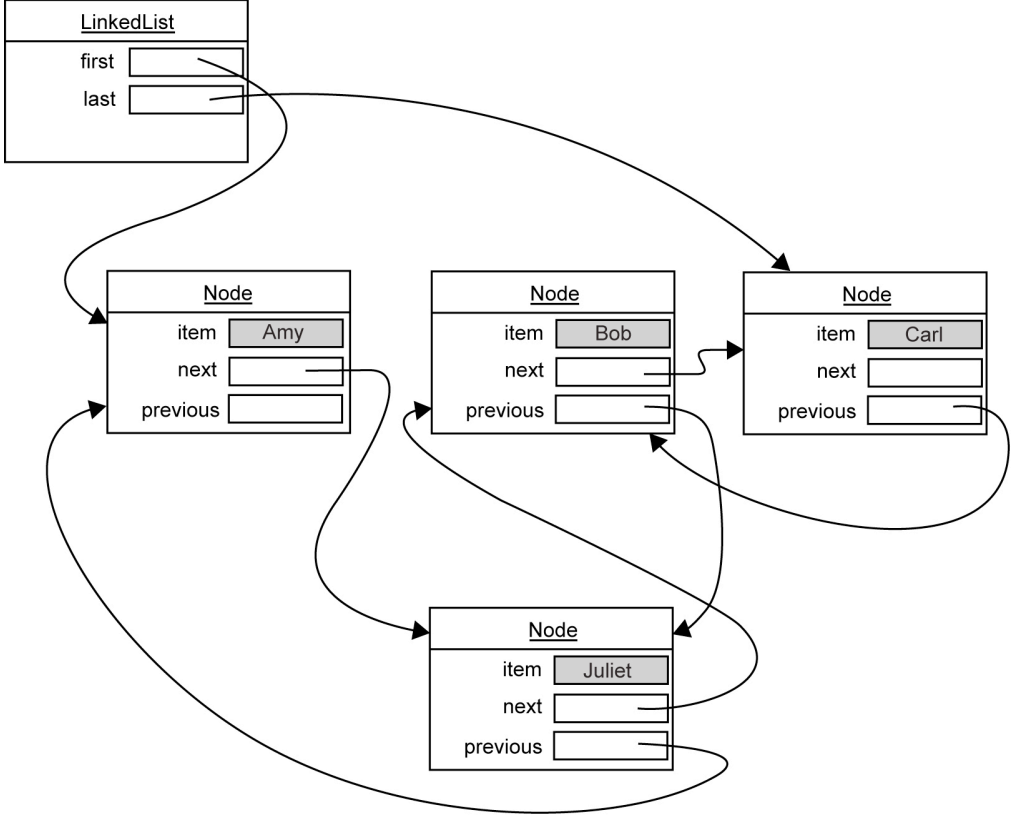


Figure 9.9: Adding an element to a linked list

If you call the `add` method multiple times, the elements are simply added in the order in which you supplied them. They are all added in turn before the current iterator position.

When you use the `add` operation with an iterator that was freshly returned from the `listIterator` method and that points to the first element of the list, the newly added element becomes the first element. When the iterator has passed the last element of the list (that is, when `hasNext` returns `false`), the added element becomes the new tail of the list. If the linked list has n elements, there are $n + 1$ spots for adding a new element. These spots correspond to the $n + 1$ possible positions of the iterator. For example, if a linked list contains three elements, A, B, and C, there are four possible positions (marked as `|`) for inserting a new element:

```
|ABC
A|BC
AB|C
ABC|
```



Note: Be careful with the "cursor" analogy. The `remove` operation does not work exactly like the Backspace key. Immediately after a call to `next`, the `remove` method indeed removes the element to the left of the iterator, just like the Backspace key would. However, if you have just called `previous`, the element to the right will be removed. And you can't call `remove` twice in a row.

Unlike the `add` method, which depends only on the iterator position, the `remove` method depends on the iterator state.

Finally, a `set` method replaces the last element, returned by a call to `next` or `previous`, with a new element. For example, the following code replaces the first element of a list with a new value:

```
ListIterator<String> iter = list.listIterator();
String oldValue = iter.next(); // returns first element
iter.set(newValue); // sets first element to newValue
```

As you might imagine, if an iterator traverses a collection while another iterator is modifying it, confusing situations can occur. For example, suppose an iterator points before an element that another iterator has just removed. The iterator is now invalid and should no longer be used. The linked list iterators have been designed to detect such modifications. If an iterator finds that its collection has been modified by another iterator or by a method of the collection itself, it throws a `ConcurrentModificationException`. For example, consider the following code:

```
List<String> list = . . .;
ListIterator<String> iter1 = list.listIterator();
ListIterator<String> iter2 = list.listIterator();
iter1.next();
iter1.remove();
iter2.next(); // throws ConcurrentModificationException
```

The call to `iter2.next` throws a `ConcurrentModificationException` since `iter2` detects that the list was modified externally.

To avoid concurrent modification exceptions, follow this simple rule: You can attach as many iterators to a collection as you like, provided that all of them are only readers. Alternatively, you can attach a single iterator that can both read and write.

Concurrent modification detection is done in a simple way. The collection keeps track of the number of mutating operations (such as adding and removing elements). Each iterator keeps a separate count of the number of mutating operations that *it* was responsible for. At the beginning of each iterator method, the iterator simply checks whether its own mutation count equals that of the collection. If not, it throws a `ConcurrentModificationException`.



Note: There is, however, a curious exception to the detection of concurrent modifications. The linked list only keeps track of *structural* modifications to the list, such as adding and removing links. The `set` method does *not* count as a structural modification. You can attach multiple iterators to a

linked list, all of which call `set` to change the contents of existing links. This capability is required for a number of algorithms in the `Collections` class that we discuss later in this chapter.

Now you have seen the fundamental methods of the `LinkedList` class. Use a `ListIterator` to traverse the elements of the linked list in either direction and to add and remove elements.

As you saw in **Section 9.2**, many other useful methods for operating on linked lists are declared in the `Collection` interface. These are, for the most part, implemented in the `AbstractCollection` superclass of the `LinkedList` class. For example, the `toString` method invokes `toString` on all elements and produces one long string of the format `[A, B, C]`. This is handy for debugging. Use the `contains` method to check whether an element is present in a linked list. For example, the call `staff.contains("Harry")` returns `true` if the linked list already contains a string equal to the string `"Harry"`.

The Java Collections Framework also supplies a number of methods that are, from a theoretical perspective, somewhat dubious. Linked lists do not support fast random access. If you want to see the nth element of a linked list, you have to start at the beginning and skip past the first $n - 1$ elements. There is no shortcut. For that reason, programmers don't usually use linked lists in situations where elements need to be accessed by an integer index.

Nevertheless, the `LinkedList` class supplies a `get` method that lets you access a particular element:

```
LinkedList<String> list = . . .;
String obj = list.get(n);
```

Of course, this method is not very efficient. If you find yourself using it, you are probably using a wrong data structure for your problem.

You should *never* use this illusory random access method to step through a linked list. The code

```
for (int i = 0; i < list.size(); i++)
    do something with list.get(i);
```

is staggeringly inefficient. Each time you look up another element, the search starts again from the beginning of the list. The `LinkedList` object makes no effort to cache the position information.



Note: The `get` method has one slight optimization: If the index is at least `size() / 2`, the search for the element starts at the end of the list.

The list iterator interface also has a method to tell you the index of the current position. In fact, since Java iterators conceptually point between elements, it has two of them: The `nextIndex` method returns the integer index of the element that would be returned by the next call to `next`; the `previousIndex` method returns the index of the element that would be returned by the next call to `previous`. Of course, that is simply one less than `nextIndex`. These methods are efficient for `LinkedList` iterators—each iterator keeps a count of its current position. Finally, if you have an integer index `n`, then `list.listIterator(n)` returns an iterator that points just before the element with index `n`. That is, calling `next` yields the same element as `list.get(n)`; obtaining that iterator is inefficient.

If you have a linked list with only a handful of elements, you don't have to be overly paranoid about the cost of the `get` and `set` methods. But then, why use a linked list in the first place? The only reason to use a linked list is to minimize the cost of insertion and removal in the middle of the list. If you have only a few elements, you can just use an `ArrayList`.

I recommend that you simply stay away from all methods that use an integer index to denote a position in a linked list. If you want random access into a collection, use an array or `ArrayList`, not a linked list.

The program in **Listing 9.1** puts linked lists to work. It simply creates two lists, merges them, then removes every second element from the second list, and finally tests the `removeAll` method. I recommend that you trace the program flow and pay special attention to the iterators. You may find it helpful to draw diagrams of the iterator positions, like this:

```
|ACE  |BDFG
A|CE  |BDFG
AB|CE B|DFG
. . .
```

Note that the call

```
IO.println(a);
```

prints all elements in the linked list `a` by invoking the `toString` method in `AbstractCollection`.

Listing 9.1 v1ch09/LinkedListDemo.java

```
1  package v1ch09;
2
3  import module java.base;
4
5  /**
```

```
 * This program demonstrates operations on linked lists.
 */
class LinkedListDemo {
    void main() {
        var a = new LinkedList<String>();
        a.add("Amy");
        a.add("Carl");
        a.add("Erica");

        var b = new LinkedList<String>();
        b.add("Bob");
        b.add("Doug");
        b.add("Frances");
        b.add("Gloria");

        // merge the elements from b into a

        ListIterator<String> aIter = a.listIterator();
        Iterator<String> bIter = b.iterator();

        while (bIter.hasNext()) {
            if (aIter.hasNext()) aIter.next();
            aIter.add(bIter.next());
        }

        IO.println(a);

        // remove every second element from b

        bIter = b.iterator();
        while (bIter.hasNext()) {
            bIter.next(); // skip one element
            if (bIter.hasNext()) {
                bIter.next(); // skip next element
                bIter.remove(); // remove that element
            }
        }

        IO.println(b);

        // bulk operation: remove all elements in b from a

        a.removeAll(b);

        IO.println(a);
    }
}
```

java.util.List<E> **1.2**

- `ListIterator<E> listIterator()`
  returns a list iterator for visiting the elements of the list.

- `ListIterator<E> listIterator(int index)`
  returns a list iterator for visiting the elements of the list whose first call to `next` will return the element with the given index.
- `void add(int i, E element)`
  adds an element at the specified position.
- `boolean addAll(int i, Collection<? extends E> elements)`
  adds all elements from a collection to the specified position.
- `E remove(int i)`
  removes and returns the element at the specified position.
- `E get(int i)`
  gets the element at the specified position.
- `E set(int i, E element)`
  replaces the element at the specified position with a new element and returns the old element.
- `int indexOf(Object element)`
  returns the position of the first occurrence of an element equal to the specified element, or `-1` if no matching element is found.
- `int lastIndexOf(Object element)`
  returns the position of the last occurrence of an element equal to the specified element, or `-1` if no matching element is found.

java.util.ListIterator<E> **1.2**

- `void add(E newElement)`
  adds an element before the current position.
- `void set(E newElement)`
  replaces the last element visited by `next` or `previous` with a new element. Throws an `IllegalStateException` if the list structure was modified since the last call to `next` or `previous`.
- `boolean hasPrevious()`
  returns `true` if there is another element to visit when iterating backwards through the list.
- `E previous()`
  returns the previous object. Throws a `NoSuchElementException` if the beginning of the list has been reached.
- `int nextIndex()`
  returns the index of the element that would be returned by the next call to `next`, or the size of the list if the iterator is past the last element.
- `int previousIndex()`
  returns the index of the element that would be returned by the next call to `previous`, or -1 if the iterator is before the first element.

`java.util.LinkedList<E>` **1.2**

- `LinkedList()`
  constructs an empty linked list.

- `LinkedList(Collection<? extends E> elements)`
  constructs a linked list and adds all elements from a collection.
- `void addFirst(E element)`
- `void addLast(E element)`
  add an element to the beginning or the end of the list.
- `E getFirst()`
- `E getLast()`
  return the element at the beginning or the end of the list.
- `E removeFirst()`
- `E removeLast()`
  remove and return the element at the beginning or the end of the list.

9.3.2. Array Lists

In the preceding section, you saw the `List` interface and the `LinkedList` class that implements it. The `List` interface describes an ordered collection in which the position of elements matters. There are two protocols for visiting the elements: through an iterator and by random access with methods `get` and `set`. The latter is not appropriate for linked lists, but of course `get` and `set` make a lot of sense for arrays. The Java Collections Framework supplies the familiar `ArrayList` class that also implements the `List` interface. An `ArrayList` encapsulates a dynamically reallocated array of objects.



Note: Java 1.0 came with a different array-based collection, the `Vector` class. All methods of the `Vector` class are *synchronized*. It is safe to access a `Vector` object from two threads. But if you access a vector from only a single thread—by far the more common case—the synchronization is wasteful. In contrast, the `ArrayList` methods are not synchronized. As you will see in **Chapter 10**, there are now better threadsafe choices as well. There is no reason to use `Vector` nowadays unless you need to interface with an ancient API.

9.3.3. Hash Sets

Linked lists and arrays let you specify the order in which you want to arrange the elements. However, if you are looking for a particular element and don't remember its position, you need to visit all elements until you find a match. That can be time consuming if the collection contains many elements. If you don't care about the ordering of the elements, there are data structures that let you find elements much faster. The drawback is that those data structures give you no control over the order in which the elements appear. These data structures organize the elements in an order that is convenient for their own purposes.

A well-known data structure for finding objects quickly is the *hash table*. A hash table computes an integer, called the *hash code*, for each object. A hash code is somehow derived from the instance fields of an object, preferably in such a way

that objects with different data yield different codes. **Table 9.2** lists a few examples of hash codes that result from the `hashCode` method of the `String` class.

Table 9.2: Hash Codes Resulting from the `hashCode` Method

String	Hash Code
`"Lee"`	`76268`
`"lee"`	`107020`
`"eel"`	`100300`

If you define your own classes, you are responsible for implementing your own `hashCode` method—see **Chapter 5** for more information. Your implementation needs to be compatible with the `equals` method: If `a.equals(b)`, then `a` and `b` must have the same hash code.

What's important for now is that hash codes can be computed quickly and that the computation depends only on the state of the object that needs to be hashed, not on the other objects in the hash table.

In Java, hash tables are implemented as arrays of linked lists. Each list is called a *bucket* (see **Figure 9.10**). To find the place of an object in the table, compute its hash code and reduce it modulo the total number of buckets. The resulting number is the index of the bucket that holds the element. For example, if an object has hash code 76268 and there are 128 buckets, then the object is placed in bucket 108 (because the remainder 76268 `%` 128 is 108). Perhaps you are lucky and there is no other element in that bucket. Then, you simply insert the element into that bucket. Of course, sometimes you will hit a nonempty bucket. This is called a *hash collision*. Then, compare the new object with all objects in that bucket to see if it is already present. If the hash codes are reasonably randomly distributed and the number of buckets is large enough, only a few comparisons should be necessary.

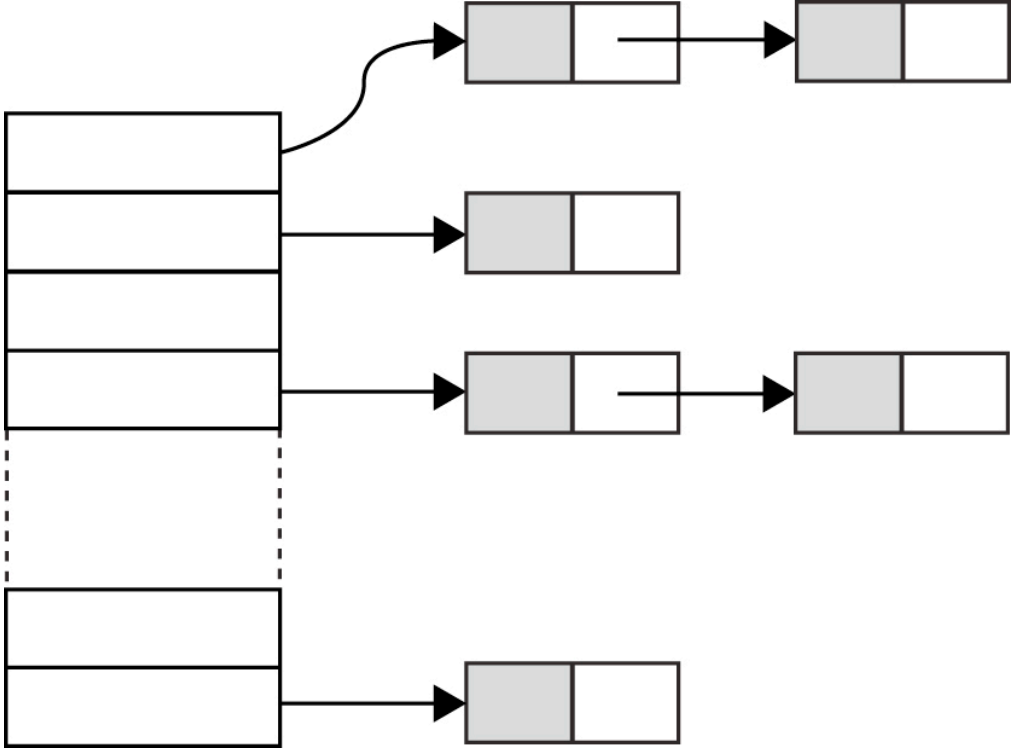


Figure 9.10: A hash table



Note: In the `HashMap` implementation, a bucket changes from a linked list into a balanced binary tree when its size exceeds a threshold. This improves performance if a hash function was poorly chosen and yields many collisions, or if malicious code tries to flood a hash table with many values that have identical hash codes.



Tip: If at all possible, the keys of a hash table should belong to a class that implements the `Comparable` interface. Then you are guaranteed not to suffer poor performance due to poorly distributed hash codes.

If the hash table gets too full, it needs to be *rehashed*. To rehash the table, a table with more buckets is created, all elements are inserted into the new table, and the original table is discarded. The *load factor* determines when a hash table is rehashed. For example, if the load factor is 0.75 (which is the default) and the table is more than 75% full, it is automatically rehashed with twice as many buckets. For most applications, it is reasonable to leave the load factor at 0.75.

If you want more control over the performance of the hash table, you can specify the initial bucket count. You should set it to the expected element count, divided by the load factor. Or, as of Java 19, you can use the static `newHashSet` convenience method that does this computation for you:

```
HashSet<String> strings = HashSet.newHashSet(expectedElementCount);
```



Note: Some researchers believe that it is a good idea to make the bucket count a prime number to prevent a clustering of keys. The evidence for this isn't conclusive, however. The Java Collections Framework uses bucket

counts that are powers of 2, with a default of `16`. (Any value you supply for the bucket count is automatically rounded to the next power of 2.)

Hash tables can be used to implement several important data structures. The simplest among them is the *set* type. A set is a collection of elements without duplicates. The `add` method of a set first tries to find the object to be added, and adds it only if it is not yet present.

The Java Collections Framework supplies a `HashSet` class that implements a set based on a hash table. You add elements with the `add` method. The `contains` method is redefined to make a fast lookup to see if an element is already present in the set. It checks only the elements in one bucket and not all elements in the collection.

The hash set iterator visits all buckets in turn. Since hashing scatters the elements around in the table, they are visited in a seemingly random order. You would only use a `HashSet` if you don't care about the ordering of the elements in the collection.

The sample program at the end of this section (**Listing 9.2**) reads words from a file, adds them to a set, and prints out the first twenty words in the set. For example, you can feed the program the text from *Alice in Wonderland* by launching it from a command shell as

```
java v1ch09.set.SetDemo gutenberg/alice30.txt
```

Alice in Wonderland has 28,195 words, of which 5,909 are unique, including the copyright notice at the beginning. The words appear in random order. The same words are then inserted into a tree set, which, as you will see in the following section, keeps the elements in sorted order.



Caution: Be careful when you mutate set elements. If the hash code of an element were to change, the element would no longer be in the correct position in the data structure.

```
var rects = new HashSet<Rectangle>();
var rect = new Rectangle(5, 10, 20, 30);
rects.add(rect);
rect.setLocation(0, 0);
    // rects is now [java.awt.Rectangle[x=0,y=0,width=20,height=30]]
rects.remove(new Rectangle(0, 0, 20, 30)); // Returns false, doesn't remove
```

Listing 9.2 v1ch09/SetDemo.java

```
package v1ch09;

import module java.base;

/**
 * This program compares insertion into a hash set and a tree set. You can specify
 * a filename and a repetition count on the command line.
 */
class SetDemo {
    void main(String[] args) throws Exception {
        List<String> words = new ArrayList<>();
        String filename = args.length > 0 ? args[0] : "gutenberg/crsto10.txt";
        int repetitions = args.length > 1 ? Integer.parseInt(args[1]) : 1;
        try (var in = new Scanner(Path.of(filename))) {
            while (in.hasNext()) {
                String word = in.next();
                words.add(word);
            }
        }
        time(new HashSet<>(), words, repetitions);
        time(new TreeSet<>(), words, repetitions);
    }

    void time(Set<String> wordSet, List<String> wordList, int repetitions) {
        long totalTime = 0;
        for (int i = 1; i <= repetitions; i++) {
            for (String word : wordList) {
                long start = System.nanoTime();
                wordSet.add(word);
                long end = System.nanoTime();
                totalTime += end - start;
            }
        }
        Iterator<String> iter = wordSet.iterator();
        for (int i = 1; i <= 20 && iter.hasNext(); i++)
            IO.print(iter.next() + " ");
        IO.println("...");
        IO.println("%s: %d words, %d distinct, %.3f seconds.".formatted(
                wordSet.getClass().getSimpleName(), wordList.size(), wordSet.size(
                totalTime * 1E-9));
    }

}
```

`java.util.HashSet<E>` 1.2

- `HashSet()`
  constructs an empty hash set.
- `HashSet(Collection<? extends E> elements)`
  constructs a hash set and adds all elements from a collection.

- `HashSet(int initialCapacity)`
  `HashSet(int initialCapacity, float loadFactor)`
  construct an empty hash set with the specified capacity and load factor, or a load factor of 0.75. If the ratio size/capacity exceeds the load factor, the hash table will be rehashed into a larger one.
- `static <E> HashSet<E> newHashSet(int numMappings)` 19
  constructs an empty hash set with sufficient initial capacity to hold the expected number of elements (`numMappings`) without rehashing.

`java.lang.Object` 1.0

- `int hashCode()`
  returns a hash code for this object. A hash code can be any integer, positive or negative. The definitions of `equals` and `hashCode` must be compatible: If `x.equals(y)` is `true`, then `x.hashCode()` must be the same value as `y.hashCode()`.

9.3.4. Tree Sets

The `TreeSet` class is similar to the hash set, with one added improvement. A tree set is a *sorted collection.* You insert elements into the collection in any order. When you iterate through the collection, the values are automatically presented in sorted order. For example, suppose you insert three strings and then visit all elements that you added.

```
var sorter = new TreeSet<String>();
sorter.add("Bob");
sorter.add("Amy");
sorter.add("Carl");
for (String s : sorter) IO.println(s);
```

Then, the values are printed in sorted order: `Amy Bob Carl`. As the name of the class suggests, the sorting is accomplished by a tree data structure. (The current implementation uses a *red-black tree*. For a detailed description of red-black trees see, for example, *Introduction to Algorithms* by Thomas Cormen, Charles Leiserson, Ronald Rivest, and Clifford Stein, The MIT Press, 2022.) Every time an element is added to a tree, it is placed into its proper sorting position. Therefore, the iterator always visits the elements in sorted order.

Adding an element to a tree is slower than adding it to a hash table. But it is still much faster than checking for duplicates in an array or linked list. If the tree contains n elements, then an average of $\log_2 n$ comparisons are required to find the correct position for the new element. For example, if the tree already contains 1,000 elements, adding a new element requires about 10 comparisons.



Note: In order to use a tree set, you must be able to compare the elements. The elements must implement the `Comparable` interface, or you must supply a `Comparator` when constructing the set. (The `Comparable` and `Comparator` interfaces were introduced in **Chapter 6**.)

How much slower is a tree set? Run the program in **Listing 9.2** as

```
java v1ch09.set.SetDemo gutenberg/crsto10.txt 100
```

The Count of Monte Cristo has 466.300 words, each of which is inserted 100 times. On my test machine, the TreeSet is almost 4 times slower. If you don't need the data sorted, there is no reason to pay for the sorting overhead. More importantly, with some data it is much more difficult to come up with a sort order than a hash function. A hash function only needs to do a reasonably good job of scrambling the objects, whereas a comparison function must tell objects apart with complete precision.

To make this distinction more concrete, consider the task of collecting a set of rectangles. If you use a `TreeSet`, you need to supply a `Comparator<Rectangle>`. How do you compare two rectangles? By area? That doesn't work. You can have two different rectangles with different coordinates but the same area, and you want to keep both. The comparator must be compatible with `equals`; that is, the comparison can only be zero if the elements are equal. There is such a sort order for rectangles (the lexicographic ordering on its coordinates), but it is cumbersome to compute. In contrast, a hash function is already defined for the `Rectangle` class. It simply hashes the coordinates.



Caution: If the comparator is not compatible with the `equals` method of the element type, set equality can give inconsistent results. Here is a typical example:

```
var words = new TreeSet<String>(String.CASE_INSENSITIVE_ORDER);
```

The intent is not to collect elements that only differ in letter case:

```
commands.add("QUIT");
commands.add("quit"); // Not added
```

However, now `equals` is no longer symmetric:

```
commands.equals(Set.of("quit")) // true
Set.of("quit").equals(commands) // false
```

The `TreeSet` considers two elements equal when the comparator returns zero. But the other set uses `String.equals`.

A better approach is to use the `String.compare` ordering and only insert lowercase strings into the set.



Note: The `TreeSet` class implements the `NavigableSet` interface. That interface adds several convenient methods for locating adjacent elements. See the API notes for details.

The program in **Listing 9.3** builds two tree sets of `Item` objects. The first one is sorted by part number, the default sort order of `Item` objects. The second set is sorted by description, using a custom comparator.

Listing 9.3 v1ch09/TreeSetDemo.java

```
1  package v1ch09;
2
3  import module java.base;
4
5  /**
6   * This program sorts a set of Item objects by comparing their part numbers,
7   * then their descriptions.
8   */
9  class TreeSetDemo {
10     void main() {
11         var parts = new TreeSet<Item>();
12         parts.add(new Item("Toaster", 1234));
13         parts.add(new Item("Widget", 4562));
14         parts.add(new Item("Router", 9912));
15         IO.println(parts);
16
17         var sortByDescription = new TreeSet<Item>(Comparator.comparing(Item::descr
18
19         sortByDescription.addAll(parts);
20         IO.println(sortByDescription);
21     }
22
23     record Item(String description, int partNumber) implements Comparable<Item> {
24         public int compareTo(Item other) {
25             return Integer.compare(partNumber, other.partNumber);
26         }
27     }
28 }
```

`java.util.TreeSet<E>` 1.2

- `TreeSet()`
- `TreeSet(Comparator<? super E> comparator)`

construct an empty tree set.

- `TreeSet(Collection<? extends E> elements)`
- `TreeSet(SortedSet<E> s)`
  construct a tree set and add all elements from a collection or sorted set (in the latter case, using the same ordering).

java.util.SortedSet<E> **1.2**

- `Comparator<? super E> comparator()`
  returns the comparator used for sorting the elements, or `null` if the elements are compared with the `compareTo` method of the `Comparable` interface.
- `E first()`
- `E last()`
  return the smallest or largest element in the sorted set.

java.util.NavigableSet<E> **6**

- `E higher(E value)`
- `E lower(E value)`
  return the least element > `value` or the largest element < `value`, or `null` if there is no such element.
- `E ceiling(E value)`
- `E floor(E value)`
  return the least element ≥ `value` or the largest element ≤ `value`, or `null` if there is no such element.
- `E pollFirst()`
- `E pollLast()`
  remove and return the smallest or largest element in this set, or `null` if the set is empty.

9.3.5. Queues and Deques

As we already discussed, a queue lets you efficiently add an element after the last and remove the first element. A double-ended queue, or *deque*, lets you efficiently add or remove elements at both ends. Adding elements in the middle is not supported. The `Deque` interface is implemented by the `ArrayDeque` and `LinkedList` classes, both of which provide deques whose size grows as needed. In **Chapter 10**, you will see bounded queues and deques.

In Java 21, the methods

```
E getFirst()
E getLast()
void addFirst(E e)
void addLast(E e)
E removeFirst()
E removeLast()
```

were moved from the `Deque` interface to the `SequencedCollection` interface, which the `List` and `Set` interfaces also extend. This provides uniform methods for accessing the first and last element of a collection.

java.util.Queue<E> **5.0**

- `boolean add(E e)`
- `boolean offer(E e)`
  add the given element after the last element of this queue and return `true`, provided the queue is not full. If the queue is full, the first method throws an `IllegalStateException`, whereas the second method returns `false`.
- `E remove()`
- `E poll()`
  remove and return the first element of this queue, provided the queue is not empty. If the queue is empty, the first method throws a `NoSuchElementException`, whereas the second method returns `null`.
- `E element()`
- `E peek()`
  return the element at the head of this queue without removing it, provided the queue is not empty. If the queue is empty, the first method throws a `NoSuchElementException`, whereas the second method returns `null`.

java.util.SequencedCollection<E> **21**

- `void addFirst(E element)`
- `void addLast(E element)`
  add the given element before the first or after the last elemet of this collection. If the collection is full, throw an `IllegalStateException`.
- `E removeFirst()`
- `E removeLast()`
  remove and return the first or last element of this collection, provided the collection is not empty. If the collection is empty, throw a `NoSuchElementException`.
- `E getFirst()`
- `E getLast()`
  return the first or last element of this collection without removing it, provided the collection is not empty. If the collection is empty, throw a `NoSuchElementException`.

java.util.Deque<E> 6

- `boolean offerFirst(E element)`
- `boolean offerLast(E element)`
  add the given element before the first or after the last element of this deque. If the deque is full, return `false`.
- `E pollFirst()`
- `E pollLast()`
  remove and return the first or last element of this deque, provided the deque is not empty. If the deque is empty, return `null`.
- `E peekFirst()`
- `E peekLast()`
  return the first or last element of this deque without removing it, provided the deque is not empty. If the deque is empty, return `null`.

`java.util.ArrayDeque<E>` 6

- `ArrayDeque()`
- `ArrayDeque(int initialCapacity)`
  construct an unbounded deque with an initial capacity of 16 or the given initial capacity.

9.3.6. Priority Queues

A priority queue retrieves elements in sorted order after they were inserted in arbitrary order. That is, whenever you call the `remove` method, you get the smallest element currently in the priority queue. However, the priority queue does not sort all its elements. If you iterate over the elements, they are not necessarily sorted. The priority queue makes use of an elegant and efficient data structure called a *heap*. A heap is a self-organizing binary tree in which the `add` and `remove` operations cause the smallest element to gravitate to the root, without wasting time on sorting all elements.

Just like a `TreeSet`, a priority queue can either hold elements of a class that implements the `Comparable` interface or a `Comparator` object you supply in the constructor.

A typical use for a priority queue is job scheduling. Each job has a priority. Jobs are added in random order. Whenever a new job can be started, the highest priority job is removed from the queue. (Since it is traditional for priority 1 to be the "highest" priority, the `remove` operation yields the minimum element.)

Listing 9.4 shows a priority queue in action. Unlike iteration in a `TreeSet`, the iteration here does not visit the elements in sorted order. However, removal always yields the smallest remaining element.

Listing 9.4 v1ch09/PriorityQueueDemo.java

```
1  package v1ch09;
2
3  import module java.base;
4
5  /**
6   * This program demonstrates the use of a priority queue.
7   */
8  class PriorityQueueDemo {
9      void main() {
10         var pq = new PriorityQueue<LocalDate>();
11         pq.add(LocalDate.of(1906, 12, 9)); // G. Hopper
12         pq.add(LocalDate.of(1815, 12, 10)); // A. Lovelace
13         pq.add(LocalDate.of(1903, 12, 3)); // J. von Neumann
14         pq.add(LocalDate.of(1910, 6, 22)); // K. Zuse
15
16         IO.println("Iterating over elements . . .");
17         for (LocalDate date : pq)
18             IO.println(date);
19         IO.println("Removing elements . . .");
20         while (!pq.isEmpty())
21             IO.println(pq.remove());
22     }
23 }
```

`java.util.PriorityQueue` 5.0

- `PriorityQueue()`
- `PriorityQueue(int initialCapacity)`
  construct a priority queue for storing `Comparable` objects.
- `PriorityQueue(int initialCapacity, Comparator<? super E> c)`
  constructs a priority queue and uses the specified comparator for sorting its elements.

9.4. Maps

A set is a collection that lets you quickly find an existing element. However, to look up an element, you need to have an exact copy of the element to find. That isn't a very common lookup—usually, you have some key information, and you want to look up the associated element. The *map* data structure serves that purpose. A map stores key/value pairs. You can find a value if you provide the key. For example, you may store a table of employee records, where the keys are the employee IDs and the values are `Employee` objects. In the following sections, you will learn how to work with maps.

9.4.1. Basic Map Operations

The Java Collections Framework supplies two general-purpose implementations for maps: `HashMap` and `TreeMap`. Both classes implement the `Map` interface.

A hash map hashes the keys, and a tree map uses an ordering on the keys to organize them in a search tree. The hash or comparison function is applied *only to the keys*. The values associated with the keys are not hashed or compared.

Should you choose a hash map or a tree map? As with sets, hashing is usually a bit faster, and it is the preferred choice if you don't need to visit the keys in sorted order.

Here is how you set up a hash map for storing employees:

```
var staff = new HashMap<String, Employee>(); // HashMap implements Map
var harry = new Employee("Harry Hacker");
staff.put("987-98-9996", harry);
. . .
```

Whenever you add an object to a map, you must supply a key as well. In our case, the key is a string, and the corresponding value is an `Employee` object.

To retrieve an object, you must use (and, therefore, remember) the key.

```
var id = "987-98-9996";
Employee e = staff.get(id); // gets harry
```



Caution: For historical reasons, the `get` method is declared with a parameter of type `Object`, and not the key type. For example, the following code compiles:

```
long numericId = . . .
Employee e = staff.get(numericId); // Compiles but never gets a value
```

If no information is stored in the map with the particular key specified, `get` returns `null`.

The `null` return value can be inconvenient. Sometimes, you have a good default that can be used for keys that are not present in the map. Then use the `getOrDefault` method.

```
Map<String, Integer> scores = . . .;
int score = scores.getOrDefault(id, 0); // gets 0 if the id is not present
```

Keys must be unique. You cannot store two values with the same key. If you call the `put` method twice with the same key, the second value replaces the first one. In fact, `put` returns the previous value associated with its key argument.

The `remove` method removes an element with a given key from the map. The `size` method returns the number of entries in the map.

The easiest way of iterating over the keys and values of a map is the `forEach` method. Provide a lambda expression that receives a key and a value. That expression is invoked for each map entry in turn.

```
scores.forEach((k, v) ->
    IO.println("key=" + k + ", value=" + v));
```

Listing 9.5 illustrates a map at work. We first add key/value pairs to a map. Then, we remove one key from the map, which removes its associated value as well. Next, we change the value that is associated with a key and call the `get` method to look up a value. Finally, we iterate through the entry set.

Listing 9.5 v1ch09/MapDemo.java

```
1  package v1ch09;
2
3  import module java.base;
4
5  /**
6   * This program demonstrates the use of a map.
7   */
8  class MapDemo {
9      void main() {
10         var birthdays = new HashMap<String, LocalDate>();
11         birthdays.put("Grace Hopper", LocalDate.of(1906, 12, 9));
12         birthdays.put("Ada Lovelace", LocalDate.of(1915, 12, 10));
13         birthdays.put("John von Neumann", LocalDate.of(1903, 12, 3));
14         birthdays.put("Konrad Zuse", LocalDate.of(1910, 6, 22));
15
16         // print all entries
17         IO.println(birthdays);
18
19         // remove an entry
20         birthdays.remove("Konrad Zuse");
21
22         // replace an entry
23         birthdays.put("Ada Lovelace", LocalDate.of(1815, 12, 10));
24
25         // look up a value
26         IO.println(birthdays.get("Ada Lovelace"));
27
28         // iterate through all entries
```

```
29          birthdays.forEach((k, v) -> IO.println("key=" + k + ", value=" + v));
30      }
31  }
```

java.util.Map<K, V> 1.2

- `V get(Object key)`
  gets the value associated with the key; returns the object associated with the key, or `null` if the key is not found in the map. Implementing classes may forbid `null` keys.
- `default V getOrDefault(Object key, V defaultValue)`
  gets the value associated with the key; returns the object associated with the key, or `defaultValue` if the key is not found in the map.
- `V put(K key, V value)`
  puts the association of a key and a value into the map. If the key is already present, the new object replaces the old one previously associated with the key. This method returns the old value of the key, or `null` if the key was not previously present. Implementing classes may forbid `null` keys or values.
- `void putAll(Map<? extends K, ? extends V> entries)`
  adds all entries from the specified map to this map.
- `boolean containsKey(Object key)`
  returns `true` if the key is present in the map.
- `boolean containsValue(Object value)`
  returns `true` if the value is present in the map.
- `default void forEach(BiConsumer<? super K,? super V> action)` 8
  applies the action to all key/value pairs of this map.

`java.util.HashMap<K, V>` 1.2

- `HashMap()`
- `HashMap(int initialCapacity)`
- `HashMap(int initialCapacity, float loadFactor)`
  construct an empty hash map with the specified capacity and load factor (a number between `0.0` and `1.0` that determines at what percentage of fullness the hash table will be rehashed into a larger one). The default load factor is `0.75`.
- `static <K, V> HashMap<K, V> newHashMap(int numMappings)` 19
  constructs an empty hash map with sufficient initial capacity to hold the expected number of entries (`numMappings`) without rehashing.

`java.util.TreeMap<K,V>` 1.2

- `TreeMap()`
  constructs an empty tree map for keys that implement the `Comparable` interface.
- `TreeMap(Comparator<? super K> c)`
  constructs a tree map and uses the specified comparator for sorting its keys.
- `TreeMap(Map<? extends K, ? extends V> entries)`
  constructs a tree map and adds all entries from a map.
- `TreeMap(SortedMap<? extends K, ? extends V> entries)`
  constructs a tree map, adds all entries from a sorted map, and uses the same element comparator as the given sorted map.

java.util.SortedMap<K, V> 1.2

- `Comparator<? super K> comparator()`
  returns the comparator used for sorting the keys, or `null` if the keys are compared with the `compareTo` method of the `Comparable` interface.
- `K firstKey()`
- `K lastKey()`
  return the smallest or largest key in the map.

9.4.2. Updating Map Entries

A tricky part of dealing with maps is updating an entry. Normally, you get the old value associated with a key, update it, and put back the updated value. But you have to worry about the special case of the first occurrence of a key. Consider using a map for counting how often a word occurs in a file. When we see a word, we'd like to increment a counter like this:

```
counts.put(word, counts.get(word) + 1);
```

That works, except in the case when `word` is encountered for the first time. Then `get` returns `null`, and a `NullPointerException` occurs.

A simple remedy is to use the `getOrDefault` method:

```
counts.put(word, counts.getOrDefault(word, 0) + 1);
```

Another approach is to first call the `putIfAbsent` method. It only puts a value if the key was previously absent (or mapped to `null`).

```
counts.putIfAbsent(word, 0);
counts.put(word, counts.get(word) + 1); // now we know that get will succeed
```

But you can do better than that. The `merge` method simplifies this common operation. The call

```
counts.merge(word, 1, Integer::sum);
```

associates `word` with `1` if the key wasn't previously present, and otherwise combines the previous value and `1`, using the `Integer::sum` function.

Now consider another common situation. We want to associate a set with each key. For example, in a book index, each term has a set of page numbers where the term occurs. Here is how to update the map:

```
var index = new TreeMap<String, TreeSet<Integer>>();
. . .
index.computeIfAbsent(term, _ -> new TreeSet<>()).add(pageNumber);
```

Here, it is better to use `computeIfAbsent` because the `TreeSet` is only constructed when there was no prior set associated with the term. Conveniently, `computeIfAbsent` returns the new value, so that we can chain the call to `add`. In contrast, `putIfAbsent` returns the previous key or `null`, which is not useful for chaining.



Caution: It is tempting to try changing `k -> new TreeSet<>()` into `TreeSet::new`, but that does not work. The function has a parameter `k` for the given key, which would be passed on to the constructor. Fortunately, there is no `TreeSet` constructor with a `String` parameter, and the code does not compile.

It could have been worse. Suppose you have a `Map<Integer, ArrayList<String>>` and call

```
map.computeIfAbsent(n, ArrayList::new)
```

There *is* an `ArrayList` constructor that takes an integer capacity. For large values of `n`, large array lists would be allocated, which was surely not intended.



Caution: The map methods are not very consistent about `null` values. Some methods treat `null` as a valid value, but others consider it in the same way as a missing key.

The `getOrDefault` method falls in the former camp, and `putIfAbsent` in the latter:

```
counts.put("C++", null);
counts.getOrDefault("C++", -1) // Yields null without using the default
counts.putIfAbsent("C++", 1) // Puts 1, interpreting null as absent
```

The API notes describe other methods for updating map entries that are less commonly used.

java.util.Map<K, V> **1.2**

- `default V merge(K key, V value, BiFunction<? super V,? super V,? extends V> remappingFunction)` **8**
  If `key` is associated with a non-`null` value `v`, applies the function to `v` and `value` and either associates `key` with the result or, if the result is `null`, removes the key. Otherwise, associates `key` with `value`. Returns `get(key)`.
- `default V compute(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)` **8**
  Applies the function to `key` and `get(key)`. Either associates `key` with the result or, if the result is `null`, removes the key. Returns `get(key)`.
- `default V computeIfPresent(K key, BiFunction<? super K,? super V,? extends V> remappingFunction)` **8**
  If `key` is associated with a non-`null` value `v`, applies the function to `key` and `v` and either associates `key` with the result or, if the result is `null`, removes the key. Returns `get(key)`.
- `default V computeIfAbsent(K key, Function<? super K,? extends V> mappingFunction)` **8**
  Applies the function to `key` unless `key` is associated with a non-`null` value. Either associates `key` with the result or, if the result is `null`, removes the key. Returns `get(key)`.
- `default void replaceAll(BiFunction<? super K,? super V,? extends V> function)` **8**
  Calls the function on all entries. Associates keys with non-`null` results and removes keys with `null` results.
- `default V putIfAbsent(K key, V value)` **8**
  If `key` is absent or associated with `null`, associates it with `value` and returns `null`. Otherwise returns the associated value.

9.4.3. Map Views

The Java Collections Framework does not consider a map itself as a collection. (Other frameworks for data structures consider a map as a collection of key/value pairs, or as a collection of values indexed by the keys.) However, you can obtain *views* of the map—objects that implement the `Collection` interface or one of its subinterfaces.

There are three views: the set of keys, the collection of values (which is not a set), and the set of key/value pairs. The keys and key/value pairs form a set because there can be only one copy of a key in a map. The methods

```
Set<K> keySet()
Collection<V> values()
Set<Map.Entry<K, V>> entrySet()
```

return these three views. (The elements of the entry set are objects of a class implementing the `Map.Entry` interface.)

Note that the `keySet` is *not* a `HashSet` or `TreeSet`, but an object of some other class that implements the `Set` interface. The `Set` interface extends the `Collection` interface. Therefore, you can use a `keySet` as you would use any collection.

For example, you can enumerate all keys of a map:

```
Set<String> keys = map.keySet();
for (String key : keys) {
    do something with key
}
```

If you want to look at both keys and values, you can avoid value lookups by enumerating the *entries*. Use the following code skeleton:

```
for (Map.Entry<String, Integer> entry : counts.entrySet()) {
    String k = entry.getKey();
    Integer v = entry.getValue();
    do something with k, v
}
```

The `Map.Entry` instances are connected to the map. You can use an entry object to update a value in the underlying map.

```
for (Map.Entry<String, Integer> entry : counts.entrySet()) {
     String k = entry.getKey();
     Integer v = entry.getValue();
     entry.setValue(v + 1); // same as counts.put(k, v + 1);
}
```

Conversely, if you update a value through other means (for example, by calling the map's `put` method), then the entry is also updated.

If you want to pass entries to some other method, you should disassociate them from the map by calling `Map.Entry.copyOf(entry)`. You can also create unas-

sociated `Map.Entry` instances by calling `Map.entry(key, value)`. This is handy whenever you need a pair of values.



Tip: You can avoid the cumbersome `Map.Entry` by using a `var` declaration.

```
for (var entry : map.entrySet()) {
    do something with entry.getKey(), entry.getValue()
}
```

Or simply use the `forEach` method:

```
map.forEach((k, v) -> {
    do something with k, v
});
```

If you invoke the `remove` method of the iterator on the key set view, you actually remove the key *and its associated value* from the map. However, you cannot *add* an element to the key set view. It makes no sense to add a key without also adding a value. If you try to invoke the `add` method, it throws an `UnsupportedOperationException`. The entry set view has the same restriction, even though it would make conceptual sense to add a new key/value pair.

java.util.Map<K, V> **1.2**

- `Set<Map.Entry<K, V>> entrySet()`
  returns a set view of `Map.Entry` objects, the key/value pairs in the map. You can remove elements from this set and they are removed from the map, but you cannot add any elements.
- `Set<K> keySet()`
  returns a set view of all keys in the map. You can remove elements from this set and the keys and associated values are removed from the map, but you cannot add any elements.
- `Collection<V> values()`
  returns a collection view of all values in the map. You can remove elements from this collection and the removed value and its key are removed from the map, but you cannot add any elements.

java.util.Map.Entry<K, V> **1.2**

- `K getKey()`
- `V getValue()`
  return the key or value of this entry.

- `V setValue(V newValue)`
  changes the value *in the associated map* to the new value and returns the old value.
- `static <K, V> Map.Entry<K,V> copyOf(Map.Entry<? extends K,? extends V> map)` [17]
  yields a copy of the given map entry. Unlike the elements of a map's entry set, the copy is not "live." Calling `setValue` does not update any map.

9.4.4. Weak Hash Maps

The Java Collections Framework has several map classes for specialized needs that we briefly discuss in this and the following sections.

The `WeakHashMap` class was designed to solve an interesting problem. What happens with a value whose key is no longer used anywhere in your program? Suppose the last reference to a key has gone away. Then, there is no longer any way to refer to the value object. But, as no part of the program has the key any more, the key/value pair cannot be removed from the map. Why can't the garbage collector remove it? Isn't it the job of the garbage collector to remove unused objects?

Unfortunately, it isn't quite so simple. The garbage collector traces *live* objects. As long as the map object is live, *all* buckets in it are live and won't be reclaimed. Thus, your program should take care to remove unused values from long-lived maps. Or, you can use a `WeakHashMap` instead. This data structure cooperates with the garbage collector to remove key/value pairs when the only reference to the key is the one from the hash table entry.

Here are the inner workings of this mechanism. The `WeakHashMap` uses *weak references* to hold keys. A `WeakReference` object holds a reference to another object—in our case, a hash table key. Objects of this type are treated in a special way by the garbage collector. Normally, if the garbage collector finds that a particular object has no references to it, it simply reclaims the object. However, if the object is reachable *only* by a `WeakReference`, the garbage collector still reclaims the object, but places the weak reference that led to it into a queue. The operations of the `WeakHashMap` periodically check that queue for newly arrived weak references. The arrival of a weak reference in the queue signifies that the key was no longer used by anyone and has been collected. The `WeakHashMap` then removes the associated entry.

9.4.5. Linked Hash Sets and Maps

The `LinkedHashSet` and `LinkedHashMap` classes remember in which order you inserted items. That way, you can avoid the seemingly random order of items in a hash table. As entries are inserted into the table, they are joined in a doubly linked list (see **Figure 9.11**).

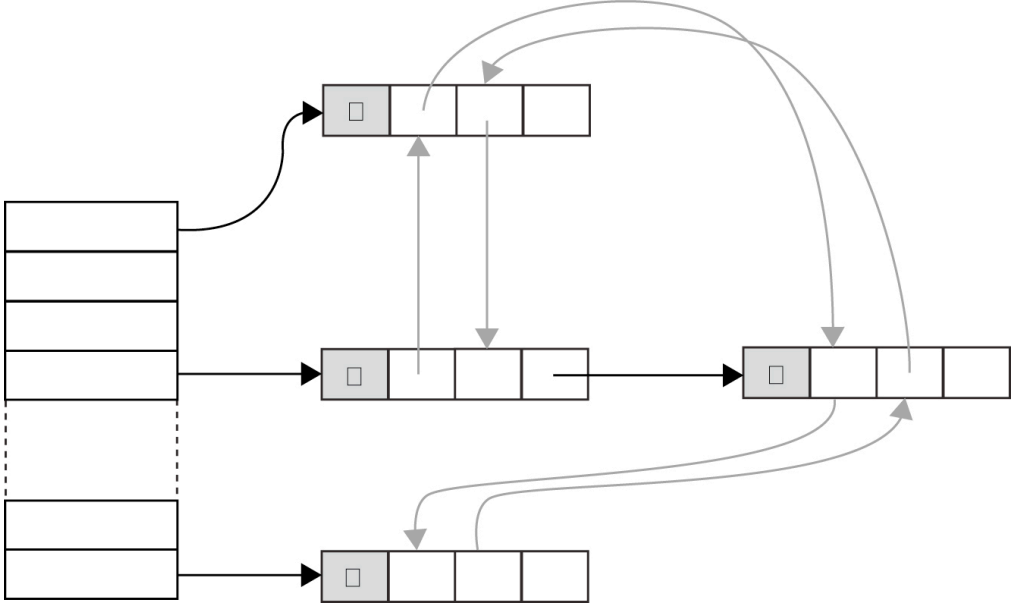


Figure 9.11: A linked hash table

For example, consider the following map insertions from **Listing 9.5**:

```
var staff = new LinkedHashMap<String, Employee>();
staff.put("144-25-5464", new Employee("Amy Lee"));
staff.put("567-24-2546", new Employee("Harry Hacker"));
staff.put("157-62-7935", new Employee("Gary Cooper"));
staff.put("456-62-5527", new Employee("Francesca Cruz"));
```

Then, `staff.keySet().iterator()` enumerates the keys in this order:

```
144-25-5464
567-24-2546
157-62-7935
456-62-5527
```

and `staff.values().iterator()` enumerates the values in this order:

```
Amy Lee
Harry Hacker
Gary Cooper
Francesca Cruz
```

A linked hash map can alternatively use *access order*, not insertion order, to iterate through the map entries. Every time you call `get` or `put`, the affected entry is removed from its current position and placed at the *end* of the linked list of entries. (Only the position in the linked list of entries is affected, not the hash table bucket. An entry always stays in the bucket that corresponds to the hash code of the key.) To construct such a hash map, call

```
LinkedHashMap<K, V>(initialCapacity, loadFactor, true)
```

Access order is useful for implementing a “least recently used” discipline for a cache. For example, you may want to keep frequently accessed entries in memory and read less frequently accessed objects from a database. When you don’t find an entry in the table, and the table is already pretty full, you can get an iterator into the table and remove the first few elements that it enumerates. Those entries were the least recently used ones.

You can even automate that process. Form a subclass of `LinkedHashMap` and override the method

```
protected boolean removeEldestEntry(Map.Entry<K, V> eldest)
```

This method is called after adding a new entry. If the method returns `true`, the `eldest` entry is removed. For example, the following cache is kept at a size of at most 100 elements:

```
var cache = new LinkedHashMap<K, V>(128, 0.75F, true) {
    protected boolean removeEldestEntry(Map.Entry<K, V> eldest) {
        return size() > 100;
    }
};
```

You can inspect the `eldest` entry to decide whether to remove it. For example, you may want to check a time stamp stored with `eldest` and only ask for removal if it is sufficiently old. This will not automatically remove all old elements since the `removeEldestEntry` method is only called once for each new entry. You are allowed to modify the map in the `removeEldestEntry` method, for example by removing the initial elements that are sufficiently old. In that case, you *must* return `false`.

If you need a set of least recently used elements instead of a map, see **Section 9.5.4**.

9.4.6. Enumeration Sets and Maps

The `EnumSet` is an efficient set implementation with elements that belong to an enumerated type. Since an enumerated type has a finite number of instances, the `EnumSet` is internally implemented simply as a sequence of bits. A bit is turned on if the corresponding value is present in the set.

The `EnumSet` class has no public constructors. Use a static factory method to construct the set:

```
enum Weekday { MONDAY, TUESDAY, WEDNESDAY, THURSDAY, FRIDAY, SATURDAY, SUNDAY };
EnumSet<Weekday> always = EnumSet.allOf(Weekday.class);
EnumSet<Weekday> never = EnumSet.noneOf(Weekday.class);
```

```
EnumSet<Weekday> workday = EnumSet.range(Weekday.MONDAY, Weekday.FRIDAY);
EnumSet<Weekday> mwf = EnumSet.of(Weekday.MONDAY, Weekday.WEDNESDAY, Weekday.FRIDAY);
```

You can use the usual methods of the `Set` interface to modify an `EnumSet`.

An `EnumMap` is a map with keys that belong to an enumerated type. It is simply and efficiently implemented as an array of values. You need to specify the key type in the constructor:

```
var personInCharge = new EnumMap<Weekday, Employee>(Weekday.class);
```



Note: In the API documentation for `EnumSet` and `EnumMap`, you will see odd-looking type parameters of the form `E extends Enum<E>`. This simply means "`E` is an enumerated type." All enumerated types extend the generic `Enum` class. For example, `Weekday` extends `Enum<Weekday>`.



Note: An `EnumSet` is not a `SequencedSet`. There is no fundamental reason for this restriction. Even implementing `NavigableSet` would be possible. But the Java team has not seen a use case that would warrant the effort. For the same reason, `EnumMap` is not a `SequencedMap`.

9.4.7. Identity Hash Maps

The `IdentityHashMap` has a quite specialized purpose. Here, the hash values for the keys should not be computed by the `hashCode` method but by the `System.identityHashCode` method. That's the method that `Object.hashCode` uses to compute a hash code from the object's memory address. Also, for comparison of objects, the `IdentityHashMap` uses `==`, not `equals`.

In other words, different key objects are considered distinct even if they have equal contents. This class is useful for implementing object traversal algorithms, such as object serialization, in which you want to keep track of which objects have already been traversed.



Caution: Java 20 fixed a subtle bug with `IdentityHashMap` that was introduced in Java 8 (https://bugs.openjdk.org/browse/JDK-8284901). Two uncommon methods falsely used `equals` instead of `==`:

```
void remove(Object key, Object value)
void replace(K key, V oldValue, V newValue)
```

These methods are sometimes used in concurrent algorithms, where a map should only be updated if it hasn't been modified by another thread. One

can only imagine the frustration of the first programmer who ran into this bug.

No software system as complex as the Java platform can be completely free from bugs. That is why it is important to regularly update to the latest version.

`java.util.WeakHashMap<K, V>` 1.2

- `WeakHashMap()`
- `WeakHashMap(int initialCapacity)`
- `WeakHashMap(int initialCapacity, float loadFactor)`
  construct an empty hash map with the specified capacity and load factor.

`java.util.LinkedHashSet<E>` 1.4

- `LinkedHashSet()`
- `LinkedHashSet(int initialCapacity)`
- `LinkedHashSet(int initialCapacity, float loadFactor)`
  construct an empty linked hash set with the specified capacity and load factor.

`java.util.LinkedHashMap<K, V>` 1.4

- `LinkedHashMap()`
- `LinkedHashMap(int initialCapacity)`
- `LinkedHashMap(int initialCapacity, float loadFactor)`
- `LinkedHashMap(int initialCapacity, float loadFactor, boolean accessOrder)`
  construct an empty linked hash map with the specified capacity, load factor, and ordering. The `accessOrder` parameter is `true` for access order, `false` for insertion order.
- `protected boolean removeEldestEntry(Map.Entry<K, V> eldest)`
  should be overridden to return `true` if you want the `eldest` entry to be removed. The `eldest` parameter is the entry whose removal is being contemplated. This method is called after an entry has been added to the map. The default implementation returns `false`—old elements are not removed by default. However, you can redefine this method to selectively return `true`—for example, if the eldest entry fits a certain condition or if the map exceeds a certain size.

`java.util.EnumSet<E extends Enum<E>>` 5.0

- `static <E extends Enum<E>> EnumSet<E> allOf(Class<E>`

`enumType)`
returns a mutable set that contains all values of the given enumerated type.

- `static <E extends Enum<E>> EnumSet<E> noneOf(Class<E> enumType)`
returns a mutable set that is initially empty.
- `static <E extends Enum<E>> EnumSet<E> range(E from, E to)`
returns a mutable set that contains all values between `from` and `to` (inclusive).
- `static <E extends Enum<E>> EnumSet<E> of(E e) . . .`
- `static <E extends Enum<E>> EnumSet<E> of(E e1, E e2, E e3, E e4, E e5)`
- `static <E extends Enum<E>> EnumSet<E> of(E first, E... rest)`
return a mutable set containing the given elements which must not be `null`.
- `public static <E extends Enum<E>> EnumSet<E> copyOf(EnumSet<E> s)`
- `public static <E extends Enum<E>> EnumSet<E> copyOf(Collection<E> c)`
create a mutable set initially containing the given elements. In the second method, `c` must either be an `EnumSet` or be nonempty (in order to determine the element type).

`java.util.EnumMap<K extends Enum<K>, V>` **5.0**

- `EnumMap(Class<K> keyType)`
constructs an empty mutable map whose keys have the given type.

`java.util.IdentityHashMap<K, V>` **1.4**

- `IdentityHashMap()`
- `IdentityHashMap(int expectedMaxSize)`
construct an empty identity hash map whose capacity is the smallest power of 2 exceeding 1.5 × `expectedMaxSize`. (The default for `expectedMaxSize` is `21`.)

`java.lang.System` **1.0**

- `static int identityHashCode(Object obj)` **1.1**
returns the same hash code (derived from the object's memory address) that `Object.hashCode` computes, even if the class to which `obj` belongs has redefined the `hashCode` method.

9.5. Copies and Views

You might think it is overkill to have lots of interfaces (**Figure 9.4**) and abstract classes (**Figure 9.5**) to implement a modest number of concrete collection classes. However, these figures don't tell the whole story. By using *views*, you can obtain other objects that implement the `Collection` or `Map` interfaces. You saw one example of this with the `keySet` method of the map classes. At first glance, it appears as if the method creates a new set, fills it with all the keys of the map, and returns it. However, that is not the case. Instead, the `keySet` method returns an object of a class that implements the `Set` interface and whose methods manipulate the original map. Such a collection is called a *view*.

The technique of views has a number of useful applications in the Java Collections Framework. We will discuss these applications in the following sections.

9.5.1. Small Collections

There are static methods yielding a set or list with given elements, and a map with given key/value pairs.

For example,

```
List<String> names = List.of("Peter", "Paul", "Mary");
Set<Integer> numbers = Set.of(2, 3, 5);
```

yield a list and a set with three elements.

You can also provide an array of objects:

```
String[] namesArray = { "Peter", "Paul", "Mary" };
List<String> names = List.of(namesArray); // A list containing three objects
```



Caution: if you provide an array of primitive type values, the result is a list with a single element, namely the array:

```
int[] numbersArray = { 2, 3, 5 };
List<int[]> arrays = List.of(numbersArray);
    // A list containing a single object, an int[] array
```

There is no convenience method for turning a primitive type array into a list of its wrapped elements.

For a map, you specify the keys and values, like this:

```
Map<String, Integer> scores = Map.of("Peter", 2, "Paul", 3, "Mary", 5);
```

The elements, keys, or values may not be `null`. Set and map keys may not be duplicated:

```
numbers = Set.of(13, null); // Error--null element
scores = Map.of("Peter", 4, "Peter", 2); // Error--duplicate key
```



Caution: No guarantee is made about the iteration order of these sets and maps. In fact, the order is deliberately scrambled with a seed that is randomized at each virtual machine startup. Look at these two `jshell` runs:

```
$ jshell -q
jshell> Set.of("Peter", "Paul", "Mary")
$1 ==> [Peter, Mary, Paul]
jshell> /exit
$ jshell -q
jshell> Set.of("Peter", "Paul", "Mary")
$1 ==> [Paul, Mary, Peter]
```

Some Java programmers write programs whose correctness depends on the assumption that implementation details will never change. That can make it very difficult for the framework implementors to make useful implementation changes. In this case, the message is clear—don't write programs that assume anything about the element order.

The `List` and `Set` interfaces have eleven `of` methods with zero to ten arguments, and an `of` method with a variable number of arguments. The specializations are provided for efficiency.

For the `Map` interface, it is not possible to provide a version with variable arguments since the argument types alternate between the key and value types. There is a static method `ofEntries` that accepts an arbitrary number of `Map.Entry<K, V>` objects, which you can create with the static `entry` method. For example,

```
import static java.util.Map.*;
. . .
Map<String, Integer> scores = ofEntries(
    entry("Peter", 2),
    entry("Paul", 3),
    entry("Mary", 5));
```

The `of` and `ofEntries` methods produce objects of classes that have an instance field for each element, or that are backed by an array.

These collection objects are *unmodifiable*. Any attempt to change their contents results in an `UnsupportedOperationException`.

If you want a mutable collection, you can pass the unmodifiable collection to the constructor:

```
var names = new ArrayList<>(List.of("Peter", "Paul", "Mary")); // A mutable list of name
```

The method call

```
Collections.nCopies(n, anObject)
```

returns an immutable object that implements the `List` interface and gives the illusion of having `n` elements, each of which appears as `anObject`.

For example, the following call creates a `List` containing 100 strings, all set to `"DEFAULT"`:

```
List<String> settings = Collections.nCopies(100, "DEFAULT");
```

There is very little storage cost—the object is stored only once.



Note: The `Collections` class contains a number of utility methods with parameters or return values that are collections. Do not confuse it with the `Collection` interface.



Tip: Java doesn't have a `Pair` class, and some programmers use a `Map.Entry` as a poor man's pair.

9.5.2. Unmodifiable Copies and Views

To make an *unmodifiable copy* of a collection, use the `copyOf` method of the collection type:

```
ArrayList<String> names = . . .;
Set<String> nameSet = Set.copyOf(names); // The names as an unmodifiable set
List<String> nameList = List.copyOf(names); // The names as an unmodifiable list
```

As with the `of` methods, the `copyOf` methods refuse to create collections containing `null` elements, instead throwing a `NullPointerException`.

Each `copyOf` method makes a copy of the collection. If the original collection is modified, the copy is not affected.

If the original collection happens to be unmodifiable and of the correct type, then `copyOf` simply returns it:

```
Set<String> names = Set.of("Peter", "Paul", "Mary");
Set<String> nameSet = Set.copyOf(names); // No need to make a copy: names == nameSet
```

The `Collections` class has methods that produce *unmodifiable views* of collections. These views add a runtime check to an existing collection. If an attempt to modify the unmodifiable collection is detected, an exception is thrown.

However, if the original collection changes, the view reflects those changes. That is what makes views different from copies.

You obtain unmodifiable views by eight methods:

```
Collections.unmodifiableCollection
Collections.unmodifiableList
Collections.unmodifiableSet
Collections.unmodifiableSortedSet
Collections.unmodifiableNavigableSet
Collections.unmodifiableMap
Collections.unmodifiableSortedMap
Collections.unmodifiableNavigableMap
```

Each method is defined to work on an interface. For example, `Collections.unmodifiableList` works with an `ArrayList`, a `LinkedList`, or any other class that implements the `List` interface.

For example, suppose you want to let some part of your code look at, but not touch, the contents of a collection. Here is what you could do:

```
var staff = new LinkedList<String>();
. . .
lookAt(Collections.unmodifiableList(staff));
```

The `Collections.unmodifiableList` method returns an object of a class implementing the `List` interface. Its accessor methods retrieve values from the `staff` collection. Of course, the `lookAt` method can call all methods of the `List` interface, not just the accessors. But all mutator methods (such as `add`) have been redefined to throw an `UnsupportedOperationException` instead of forwarding the call to the underlying collection.

The unmodifiable view does not make the collection itself immutable. You can still modify the collection through its original reference (`staff`, in our case). And you can still call mutator methods on the elements of the collection.

The views wrap the *interface* and not the actual collection object, so you only have access to those methods that are defined in the interface. For example, the `LinkedList` class has convenience methods, `addFirst` and `addLast`, that are

not part of the `List` interface. These methods are not accessible through the unmodifiable view.



Caution: The `unmodifiableCollection` method (as well as the `synchronizedCollection` and `checkedCollection` methods discussed later in this section) returns a collection whose `equals` method does *not* invoke the `equals` method of the underlying collection. Instead, it inherits the `equals` method of the `Object` class, which just tests whether the objects are identical. If you turn a set or list into just a collection, you can no longer test for equal contents. The view acts in this way because equality testing is not well defined at this level of the hierarchy. The views treat the `hashCode` method in the same way.

Prefer the `unmodifiableSet` and `unmodifiableList` wrappers whose `equals` and `hashCode` methods are appropriate for sets and lists.

For example:

```
var names = Set.of("Peter", "Paul", "Mary");
Collections.unmodifiableCollection(names).equals(names) // false
Collections.unmodifiableSet(names).equals(names) // true
```

9.5.3. Subranges

You can form subrange views for a number of collections. For example, suppose you have a list `staff` and want to extract elements 10 to 19. Use the `subList` method to obtain a view into the subrange of the list:

```
List<Employee> group2 = staff.subList(10, 20);
```

The first index is inclusive, the second exclusive—just like the parameters for the `substring` operation of the `String` class.

You can apply any operations to the subrange, and they automatically reflect the entire list. For example, you can erase the entire subrange:

```
group2.clear(); // staff reduction
```

The elements get automatically cleared from the `staff` list, and `group2` becomes empty.

For sorted sets and maps, you use the sort order, not the element position, to form subranges. The `SortedSet` interface declares three methods:

```
SortedSet<E> subSet(E from, E to)
SortedSet<E> headSet(E to)
SortedSet<E> tailSet(E from)
```

These return the subsets of all elements that are larger than or equal to `from` and strictly smaller than `to`. For sorted maps, the similar methods

```
SortedMap<K, V> subMap(K from, K to)
SortedMap<K, V> headMap(K to)
SortedMap<K, V> tailMap(K from)
```

return views into the maps consisting of all entries in which the *keys* fall into the specified ranges.

The `NavigableSet` interface gives more control over these subrange operations. You can specify whether the bounds are included:

```
NavigableSet<E> subSet(E from, boolean fromInclusive, E to, boolean toInclusive)
NavigableSet<E> headSet(E to, boolean toInclusive)
NavigableSet<E> tailSet(E from, boolean fromInclusive)
```

9.5.4. Sets From Boolean-Valued Maps

Sometimes, an API provides maps with useful features that you would like in a set. In **Section 9.4.5**, you saw how you can use a `LinkedHashMap` to build a cache that discards older elements. The `LinkedHashSet` class does not have this capability.

If you want to have a set of the 100 least recently inserted strings, make a `LinkedHashMap<String, Boolean>`, and then call the `Collections.newSetFromMap` method:

```
Set<String> cache = Collections.newSetFromMap(new LinkedHashMap<String, Boolean>() {
    protected boolean removeEldestEntry(Map.Entry<String, Boolean> eldest) {
        return size() > 100;
    }
});
```

The set view is backed by the map. When you add an element `e`, the view puts an entry with key `e` and value `Boolean.TRUE` into the map. If that causes the map to have more than 100 entries, the oldest one is removed.

You should call `newSetFromMap` with an empty map. Afterwards, do not modify the map, but let all modifications occur through methods of the set view. This is best achieved by passing the map to the `newSetFromMap` method without retaining a reference to it, as in the preceding example.

The `newSetFromMap` method is also useful with `WeakHashMap` (see Section 9.4.4). With a `ConcurrentHashMap`, use the `newKeySet` method instead (see Chapter 10).

9.5.5. Reversed Views

The `reversed` methods of the `SequencedCollection` and `SequencedSet` interfaces yield views that iterate over the elements in reverse order. For example, here is how to traverse a list or tree set of strings in reverse:

```
for (String element : collection.reversed()) {
    do something with element
}
```

With `SequencedMap`, the `reversed` method views the map with the keys in reverse order.

9.5.6. Checked Views

Checked views are intended as debugging support for a problem that can occur with generic types. As explained in Chapter 8, it is actually possible to smuggle elements of the wrong type into a generic collection. For example:

```
var strings = new ArrayList<String>();
ArrayList rawList = strings; // warning only, not an error,
                             // for compatibility with legacy code
rawList.add(new Date()); // now strings contains a Date object!
```

The erroneous `add` command is not detected at runtime. Instead, a class cast exception will happen later when another part of the code calls `get` and casts the result to a `String`.

A checked view can detect this problem. Define a safe list as follows:

```
List<String> safeStrings = Collections.checkedList(strings, String.class);
```

The view's `add` method checks that the inserted object belongs to the given class and immediately throws a `ClassCastException` if it does not. The advantage is that the error is reported at the correct location:

```
ArrayList rawList = safeStrings;
rawList.add(new Date()); // checked list throws a ClassCastException
```



Caution: The checked views are limited by the runtime checks that the virtual machine can carry out. For example, if you have an

`ArrayList<Pair<String>>`, you cannot protect it from inserting a `Pair<Date>` since the virtual machine has a single "raw" `Pair` class.

9.5.7. Synchronized Views

If you access a collection from multiple threads, you need to ensure that the collection is not accidentally damaged. For example, it would be disastrous if one thread tried to add to a hash table while another thread was rehashing the elements.

Instead of implementing thread-safe collection classes, the framework designers used the view mechanism to make regular collections thread-safe. For example, the static `synchronizedMap` method in the `Collections` class can turn any map into a `Map` with synchronized access methods:

```
var map = Collections.synchronizedMap(new HashMap<String, Employee>());
```

You can now access the `map` object from multiple threads. The methods such as `get` and `put` are synchronized—each method call must be finished completely before another thread can call another method. We discuss the issue of synchronized access to data structures in greater detail in **Chapter 10**.

9.5.8. A Note on Optional Operations

A view usually has some restriction—it may be read-only, it may not be able to change the size, or it may support removal but not insertion (as is the case for the key view of a map). A restricted view throws an `UnsupportedOperationException` if you attempt an inappropriate operation.

In the API documentation for the collection and iterator interfaces, many methods are described as "optional operations." This seems to be in conflict with the notion of an interface. After all, isn't the purpose of an interface to lay out the methods that a class *must* implement? Indeed, this arrangement is unsatisfactory from a theoretical perspective. A better solution might have been to design separate interfaces for read-only views and views that can't change the size of a collection. However, that would have tripled the number of interfaces, which the designers of the framework found unacceptable.

Should you extend the technique of "optional" methods to your own designs? I think not. Even though collections are used frequently, the coding style for implementing them is not typical for other problem domains. The designers of a collection class library have to resolve a particularly brutal set of conflicting requirements. Users want the library to be easy to learn, convenient to use, completely generic, idiot-proof, and at the same time as efficient as hand-coded algorithms. It is plainly impossible to achieve all these goals simultaneously, or even to come close. But in your own programming problems, you will rarely encounter such an

extreme set of constraints. You should be able to find solutions that do not rely on the drastic measure of “optional” interface operations.

`java.util.List` 1.2

- `static <E> List<E> of()` 9
- `static <E> List<E> of(E e1)` 9
  `. . .`
- `static <E> List<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)` 9
- `static <E> List<E> of(E... elements)` 9
  yield an unmodifiable list of the given elements, which must not be `null`.
- `static <E> List<E> copyOf(Collection<? extends E> coll)` 10
  yields an unmodifiable copy of the given collection.

`java.util.Set` 1.2

- `static <E> Set<E> of()` 9
- `static <E> Set<E> of(E e1)` 9
  `. . .`
- `static <E> Set<E> of(E e1, E e2, E e3, E e4, E e5, E e6, E e7, E e8, E e9, E e10)` 9
- `static <E> Set<E> of(E... elements)` 9
  yield an unmodifiable set of the given elements, which must not be `null`.
- `static <E> Set<E> copyOf(Collection<? extends E> coll)` 10
  yields an unmodifiable copy of the given collection.

`java.util.Map` 1.2

- `static <K, V> Map<K, V> of()` 9
- `static <K, V> Map<K, V> of(K k1, V v1)` 9
  `. . .`
- `static <K,V> Map<K,V> of(K k1, V v1, K k2, V v2, K k3, V v3, K k4, V v4, K k5, V v5, K k6, V v6, K k7, V v7, K k8, V v8, K k9, V v9, K k10, V v10)` 9
  yield an unmodifiable map of the given keys and values, which must not be `null`.
- `static <K,V> Map.Entry<K,V> entry(K k, V v)` 9
  yields an unmodifiable map entry of the given key and value, which must not be `null`.
- `static <K,V> Map<K,V> ofEntries(Map.Entry<? extends K,? extends V>... entries)` 9
  yields an unmodifiable map of the given entries.

- `static <K, V> Map<K,V> copyOf(Map<? extends K,? extends V> map)` 10
  yields an unmodifiable copy of the given map.

`java.util.Collections` 1.2

- `static <E> Set<E> newSetFromMap(Map<E, Boolean> map)` 6
  yields a set backed by the given map, which should initially be empty and afterwards only accessed through the returned view.
- `static <E> Collection<E> unmodifiableCollection(Collection<E> c)`
- `static <E> SequencedCollection<E> unmodifiableSequencedCollection(SequencedCollection<E> c)` 21
- `static <E> List<E> unmodifiableList(List<E> c)`
- `static <E> Set<E> unmodifiableSet(Set<E> c)`
- `static <E> SequencedSet<E> unmodifiableSequencedSet(SequencedCollection<E> c)` 21
- `static <E> SortedSet<E> unmodifiableSortedSet(SortedSet<E> c)`
- `static <E> SortedSet<E> unmodifiableNavigableSet(NavigableSet<E> c)` 8
- `static <K, V> Map<K, V> unmodifiableMap(Map<K, V> c)`
- `static <K, V> SortedMap<K, V> unmodifiableSortedMap(SortedMap<K, V> c)`
- `static <K, V> SequencedMap<K, V> unmodifiableSequencedMap(SequencedMap<K, V> c)` 21
- `static <K, V> SortedMap unmodifiableNavigableMap(NavigableMap<K, V> c)` 8
  construct a view of the collection; the view's mutator methods throw an `UnsupportedOperationException`.
- `static <E> Collection<E> synchronizedCollection(Collection<E> c)`
- `static <E> List synchronizedList(List<E> c)`
- `static <E> Set synchronizedSet(Set<E> c)`
- `static <E> SortedSet synchronizedSortedSet(SortedSet<E> c)`
- `static <E> NavigableSet synchronizedNavigableSet(NavigableSet<E> c)` 8
- `static <K, V> Map<K, V> synchronizedMap(Map<K, V> c)`
- `static <K, V> SortedMap<K, V> synchronizedSortedMap(SortedMap<K, V> c)`
- `static <K, V> NavigableMap<K, V> synchronizedNavigableMap(NavigableMap<K, V> c)` 8
  construct a view of the collection; the view's methods are synchronized.

- `static <E> Collection checkedCollection(Collection<E> c, Class<E> elementType)`
- `static <E> List checkedList(List<E> c, Class<E> elementType)`
- `static <E> Set checkedSet(Set<E> c, Class<E> elementType)`
- `static <E> SortedSet checkedSortedSet(SortedSet<E> c, Class<E> elementType)`
- `static <E> NavigableSet checkedNavigableSet(NavigableSet<E> c, Class<E> elementType)` 8
- `static <K, V> Map checkedMap(Map<K, V> c, Class<K> keyType, Class<V> valueType)`
- `static <K, V> SortedMap checkedSortedMap(SortedMap<K, V> c, Class<K> keyType, Class<V> valueType)`
- `static <K, V> NavigableMap checkedNavigableMap(NavigableMap<K, V> c, Class<K> keyType, Class<V> valueType)` 8
- `static <E> Queue<E> checkedQueue(Queue<E> queue, Class<E> elementType)` 8

  construct a view of the collection; the view's methods throw a `ClassCastException` if an element of the wrong type is inserted.
- `static <E> List<E> nCopies(int n, E value)`

  yields an unmodifiable list with `n` identical values.
- `static <E> List<E> singletonList(E value)`
- `static <E> Set<E> singleton(E value)`
- `static <E> List<E> emptyList()`
- `static <T> Set<T> emptySet()`
- `static <E> SortedSet<E> emptySortedSet()`
- `static NavigableSet<E> emptyNavigableSet()`
- `static <K,V> Map<K,V> emptyMap()`
- `static <K,V> SortedMap<K,V> emptySortedMap()`
- `static <K,V> NavigableMap<K,V> emptyNavigableMap()`
- `static <T> Enumeration<T> emptyEnumeration()`
- `static <T> Iterator<T> emptyIterator()`
- `static <T> ListIterator<T> emptyListIterator()`

  yield an empty collection, map, or iterator.

`java.util.Arrays` 1.2

- `static <E> List<E> asList(E... array)`

  returns a list view of the elements in an array that is modifiable but not resizable.

`java.util.List<E>` 1.2

- `List<E> subList(int firstIncluded, int firstExcluded)`
  returns a list view of the elements within a range of positions.

java.util.SortedSet<E> 1.2

- `SortedSet<E> subSet(E firstIncluded, E firstExcluded)`
- `SortedSet<E> headSet(E firstExcluded)`
- `SortedSet<E> tailSet(E firstIncluded)`
  return a view of the elements within a range.

java.util.NavigableSet<E> 6

- `NavigableSet<E> subSet(E from, boolean fromIncluded, E to, boolean toIncluded)`
- `NavigableSet<E> headSet(E to, boolean toIncluded)`
- `NavigableSet<E> tailSet(E from, boolean fromIncluded)`
  return a view of the elements within a range. The `boolean` flags determine whether the bounds are included in the view.

java.util.SortedMap<K, V> 1.2

- `SortedMap<K, V> subMap(K firstIncluded, K firstExcluded)`
- `SortedMap<K, V> headMap(K firstExcluded)`
- `SortedMap<K, V> tailMap(K firstIncluded)`
  return a map view of the entries whose keys are within a range.

java.util.NavigableMap<K, V> 6

- `NavigableMap<K, V> subMap(K from, boolean fromIncluded, K to, boolean toIncluded)`
- `NavigableMap<K, V> headMap(K from, boolean fromIncluded)`
- `NavigableMap<K, V> tailMap(K to, boolean toIncluded)`
  return a map view of the entries whose keys are within a range. The `boolean` flags determine whether the bounds are included in the view.

java.util.SequencedCollection<E> 21

- `SequencedCollection<E> reversed()`
  yields a view of this sequenced collection with the elements in reverse order.

java.util.SequencedSet<E> **21**

- `SequencedSet<E> reversed()`
  yields a view of this sequenced set with the elements in reverse order.

java.util.SequencedMap<E> **21**

- `SequencedMap<E> reversed()`
  yields a view of this sequenced map with the keys in reverse order.

9.6. Algorithms

In addition to implementing collection classes, the Java Collections Framework also provides a number of useful algorithms. In the following sections, you will see how to use these algorithms and how to write your own algorithms that work well with the framework.

9.6.1. Why Generic Algorithms?

Generic collection interfaces have a great advantage—you only need to implement your algorithms once. For example, consider a simple algorithm to compute the maximum element in a collection. Traditionally, programmers would implement such an algorithm as a loop. Here is how you can find the largest element of an array:

```
if (a.length == 0) throw new NoSuchElementException();
T largest = a[0];
for (int i = 1; i < a.length; i++)
    if (largest.compareTo(a[i]) < 0)
        largest = a[i];
```

Of course, to find the maximum of an array list, you would write the code slightly differently.

```
if (v.size() == 0) throw new NoSuchElementException();
T largest = v.get(0);
for (int i = 1; i < v.size(); i++)
    if (largest.compareTo(v.get(i)) < 0)
        largest = v.get(i);
```

What about a linked list? You don't have efficient random access in a linked list, but you can use an iterator.

```
if (l.isEmpty()) throw new NoSuchElementException();
Iterator<T> iter = l.iterator();
T largest = iter.next();
```

```
while (iter.hasNext()) {
    T next = iter.next();
    if (largest.compareTo(next) < 0)
        largest = next;
}
```

These loops are tedious to write, and just a bit error-prone. Is there an off-by-one error? Do the loops work correctly for empty containers? For containers with only one element? You don't want to test and debug this code every time, but you also don't want to implement a whole slew of methods, such as these:

```
static <T extends Comparable> T max(ArrayList<T> v)
static <T extends Comparable> T max(LinkedList<T> l)
```

That's where the collection interfaces come in. Think of the *minimal* collection interface that you need to efficiently carry out the algorithm. Random access with `get` and `set` comes higher in the food chain than simple iteration. As you have seen in the computation of the maximum element in a linked list, random access is not required for this task. Computing the maximum can be done simply by iterating through the elements. Therefore, you can implement the `max` method to take *any* object that implements the `Collection` interface.

```
public static <T extends Comparable> T max(Collection<T> elements) {
    if (elements.isEmpty()) throw new NoSuchElementException();
    Iterator<T> iter = elements.iterator();
    T largest = iter.next();
    while (iter.hasNext()) {
        T next = iter.next();
        if (largest.compareTo(next) < 0)
            largest = next;
    }
    return largest;
}
```

Of course, since arrays are not collections, you need to write a separate method for arrays:

```
static <T extends Comparable> T max(T[] a)
```

Generic algorithms are a powerful concept. In fact, the standard C++ library has dozens of useful algorithms, each operating on a generic collection. The Java Collections Framework is not quite so rich, but the `Collections` and `Arrays` classes provide the basics: sorting, binary search, and some simple utility algorithms.



Note: Some useful algorithms are missing from the Java Collections Framework, but you can find them in the streams library that is covered in Volume II. For example, to find the maximum or minimum, turn a collection or array into a stream:

```
largest = coll.stream().max(Comparator.naturalOrder()).get();
```

9.6.2. Sorting and Shuffling

Computer old-timers will sometimes reminisce about how they had to use punched cards and to actually program, by hand, algorithms for sorting. Nowadays, of course, sorting algorithms are part of the standard library for most programming languages, and the Java programming language is no exception.

The `sort` method in the `Collections` class sorts a collection that implements the `List` interface.

```
var staff = new ArrayList<String>();
fill collection
Collections.sort(staff);
```

This method assumes that the list elements implement the `Comparable` interface. If you want to sort the list in some other way, you can use the `sort` method of the `List` interface and pass a `Comparator` object. Here is how you can sort a list of employees by salary:

```
Collections.sort(staff, Comparator.comparingDouble(Employee::getSalary));
```

If you want to sort a list in *descending* order, use the static convenience method `Comparator.reverseOrder()`. It returns a comparator that returns `b.compareTo(a)`. For example,

```
Collections.sort(staff, Comparator.reverseOrder());
```

sorts the elements in the list `staff` in reverse order, according to the ordering given by the `compareTo` method of the element type. Similarly,

```
Collections.sort(staff, Comparator.comparingDouble(Employee::getSalary).reversed());
```

sorts by descending salary.

You may wonder how the `sort` method sorts a list. Typically, when you look at a sorting algorithm in a book on algorithms, it is presented for arrays and uses ran-

dom element access. However, random access in a linked list is inefficient. You can actually sort linked lists efficiently by using a form of merge sort. However, the implementation in the Java programming language does not do that. It simply dumps all elements into an array, sorts the array, and then copies the sorted sequence back into the list.

The sort algorithm used in the Java Collections Framework is a bit slower than QuickSort, the traditional choice for a general-purpose sorting algorithm. However, it has one major advantage: It is *stable*, that is, it doesn't switch equal elements. Why do you care about the order of equal elements? Here is a common scenario. Suppose you have an employee list that you already sorted by name. Now you sort by salary. What happens to employees with equal salary? With a stable sort, the ordering by name is preserved. In other words, the outcome is a list that is sorted first by salary, then by name.

Collections need not implement all of their "optional" methods, so all methods with collection parameters must describe when it is safe to pass a collection to an algorithm. For example, you clearly cannot pass an `unmodifiableList` list to the `sort` algorithm. What kind of list *can* you pass? According to the documentation, the list must be modifiable but need not be resizable.

The terms are defined as follows:

- A list is *modifiable* if it supports the `set` method.
- A list is *resizable* if it supports the `add` and `remove` operations.



Note: In **Chapter 6**, you have seen the `Arrays.sort` method. There are two versions, one for sorting an array of `Comparable` instances, and another with a comparator:

```
String[] staff = . . .;
Arrays.sort(staff);
Arrays.sort(staff, Comparator.comparingDouble(Employee::getSalary));
```

The `Collections` class has an algorithm `shuffle` that does the opposite of sorting—it randomly permutes the order of the elements in a list. For example:

```
ArrayList<Card> cards = . . .;
Collections.shuffle(cards, RandomGenerator.getDefault());
```

If you supply a list that does not implement the `RandomAccess` interface, the `shuffle` method copies the elements into an array, shuffles the array, and copies the shuffled elements back into the list.

The program in **Listing 9.6** fills an array list with 49 `Integer` objects containing the numbers 1 through 49. It then randomly shuffles the list and selects the first

six values from the shuffled list. Finally, it sorts the selected values and prints them.

Listing 9.6 v1ch09/ShuffleDemo.java

```
1  package v1ch09;
2
3  import module java.base;
4
5  /**
6   * This program demonstrates the random shuffle and sort algorithms.
7   */
8  class ShuffleDemo {
9      void main() {
10         var numbers = new ArrayList<Integer>();
11         for (int i = 1; i <= 49; i++)
12             numbers.add(i);
13         Collections.shuffle(numbers, RandomGenerator.getDefault());
14         List<Integer> winningCombination = numbers.subList(0, 6);
15         Collections.sort(winningCombination);
16         IO.println(winningCombination);
17     }
18 }
```

`java.util.Collections` **1.2**

- `static <T extends Comparable<? super T>> void sort(List<T> elements)`
  sorts the elements in the list, using a stable sort algorithm. The algorithm is guaranteed to run in O(n log n) time, where n is the length of the list.
- `static void shuffle(List<?> elements)`
- `static void shuffle(List<?> elements, Random r)`
- `static void shuffle(List<?> elements, RandomGenerator r)` **21**
  randomly shuffle the elements in the list. This algorithm runs in O(na(n)) time, where n is the length of the list and a(n) is the average time to access an element.

java.util.List<E> **1.2**

- `default void sort(Comparator<? super T> comparator)` **8**
  sorts this list, using the given comparator.

java.util.Comparator<T> 1.2

- `static <T extends Comparable<? super T>> Comparator<T> reverseOrder()` 8
  yields a comparator that reverses the ordering provided by the `Comparable` interface.
- `default Comparator<T> reversed()` 8
  yields a comparator that reverses the ordering provided by this comparator.

9.6.3. Binary Search

To find an object in an array, you normally visit all elements until you find a match. However, if the array is sorted, you can look at the middle element and check whether it is larger than the element that you are trying to find. If so, keep looking in the first half of the array; otherwise, look in the second half. That cuts the problem in half, and you keep going in the same way. For example, if the array has 1024 elements, you will locate the match (or confirm that there is none) after 10 steps, whereas a linear search would have taken you an average of 512 steps if the element is present, and 1024 steps to confirm that it is not.

The `binarySearch` method of the `Collections` class implements this algorithm. Note that the collection must already be sorted, or the algorithm will return the wrong answer. Supply the collection (which must implement the `List` interface) and the target to be located. If the collection is not sorted by the `compareTo` element of the `Comparable` interface, supply a comparator object as well.

```
i = Collections.binarySearch(elements, target);
i = Collections.binarySearch(elements, target, comparator);
```

A non-negative return value from the `binarySearch` method denotes the index of the matching object. That is, `elements.get(i)` is equal to `target` under the comparison order. If the value is negative, then there is no matching element. However, you can use the return value to compute the location where you *should* insert `target` into the collection to keep it sorted. The insertion location is

```
insertionPoint = -i - 1;
```

It isn't simply `-i` because then the value of `0` would be ambiguous. In other words, the operation

```
if (i < 0)
    elements.add(-i - 1, target);
```

adds the target in the correct place.

To be worthwhile, binary search requires random access. If you have to iterate one by one through half of a linked list to find the middle element, you have lost the advantage of the binary search. The `binarySearch` algorithm reverts to an iterative algorithm of the same efficiency as linear search if you give it a list that does not implement the `RandomAccess` interface.

The `Arrays` class has binary search implementations for arrays of primitive types and objects.

`java.util.Collections` 1.2

- `static <T extends Comparable<? super T>> int binarySearch(List<T> elements, T key)`
- `static <T> int binarySearch(List<T> elements, T key, Comparator<? super T> c)`
  search for a key in a sorted list, using a binary search if the element type implements the `RandomAccess` interface, and a linear search in all other cases. The methods are guaranteed to run in O(a(n) log n) time, where n is the length of the list and a(n) is the average time to access an element. The methods return either the index of the key in the list, or a negative value `i` if the key is not present in the list. In that case, the key should be inserted at index `-i - 1` for the list to stay sorted.

`java.util.Arrays` 1.2

- `static int binarySearch(T[] a, T key)`
- `static int binarySearch(T[] a, int start, int end, T key)` 6
  use the binary search algorithm to search for the key in the sorted array `a`. If the key is found, its index is returned. Otherwise, a negative value `r` is returned; –`r` – 1 is the spot at which `key` should be inserted to keep `a` sorted. The component type `T` of the array can be `Object`, `int`, `long`, `short`, `char`, `byte`, `boolean`, `float`, or `double`.
- `static <T> T binarySearch(T[] a, T key, Comparator<? super T> c)`
- `static <T> T binarySearch(T[] a, int start, int end, T key, Comparator<? super T> c)` 6
  use the binary search algorithm to search for the key in the array `a` that has been sorted with the given comparator.

9.6.4. Simple Algorithms

The `Collections` class contains several simple but useful algorithms. Among them is the example from the beginning of this section—finding the maximum

value of a collection. Others include copying elements from one list to another, filling a container with a constant value, and reversing a list.

Why supply such simple algorithms in the framework? Surely most programmers could easily implement them with simple loops. I like the algorithms because they make life easier for the programmer *reading* the code. When you read a loop that was implemented by someone else, you have to decipher the original programmer's intentions. For example, look at this loop:

```
for (int i = 0; i < words.size(); i++)
    if (words.get(i).equals("C++")) words.set(i, "Java");
```

Now compare the loop with the call

```
Collections.replaceAll(words, "C++", "Java");
```

When you see the method call, you know right away what the code does.

The API notes at the end of this section describe the simple algorithms in the `Collections` class.

The default methods `Collection.removeIf` and `List.replaceAll` are just a bit more complex. You provide a lambda expression to test or transform elements. For example, here we remove all short words and change the remaining ones to lowercase:

```
words.removeIf(w -> w.length() <= 3);
words.replaceAll(String::toLowerCase);
```



Caution: The predicate of `removeIf` should only look at the element whose removal is being decided, and not at the collection. If you do the latter, the behavior is implementation-dependent.

The `ArrayList` version of `removeIf` method makes two passes over the elements. It first finds the elements that should be removed, and then it removes all of them. With other collections, matching elements are removed as soon as they are found.

This makes a difference if the predicate reads the collection. Consider this example, where `words` is an `ArrayList` of length 3:

```
var words = new ArrayList<String>(List.of("Ada", "C++", "Java"))
words.removeIf(w -> w.length() == words.size()); // Now words is ["Java"]
```

In the first pass, all words of length 3 are marked for removal, and in the second pass, they are removed.

However, if `words` is a `LinkedList` or a `HashSet`, then the same call to `removeIf` only removes one of the three-letter words. After the first removal, the predicate tests whether the collection has size 2.

Why the difference? A naïve one-pass implementation would have been inefficient for array-backed collections. It didn't immediately occur to the developers that there is an efficient one-pass algorithm. By the time they thought of it, they decided not to use it for compatibility's sake—see **https://bugs.openjdk.org/browse/JDK-8143577**.

`java.util.Collections` 1.2

- `static <T extends Comparable<? super T>> T min(Collection<T> elements)`
- `static <T extends Comparable<? super T>> T max(Collection<T> elements)`
- `static <T> min(Collection<T> elements, Comparator<? super T> c)`
- `static <T> max(Collection<T> elements, Comparator<? super T> c)`
  return the smallest or largest element in the collection. (The parameter bounds are simplified for clarity.)
- `static <T> void copy(List<? super T> to, List<T> from)`
  copies all elements from a source list to the same positions in the target list. The target list must be at least as long as the source list.
- `static <T> void fill(List<? super T> l, T value)`
  sets all positions of a list to the same value.
- `static <T> boolean addAll(Collection<? super T> c, T... values)` 5.0
  adds all values to the given collection and returns `true` if the collection changed as a result.
- `static <T> boolean replaceAll(List<T> l, T oldValue, T newValue)` 1.4
  replaces all elements equal to `oldValue` with `newValue`.
- `static int indexOfSubList(List<?> l, List<?> s)` 1.4
- `static int lastIndexOfSubList(List<?> l, List<?> s)` 1.4
  return the index of the first or last sublist of `l` equaling `s`, or `-1` if no sublist of `l` equals `s`. For example, if `l` is `[s, t, a, r]` and `s` is `[t, a, r]`, then both methods return the index `1`.
- `static void swap(List<?> l, int i, int j)` 1.4
  swaps the elements at the given offsets.

- `static void reverse(List<?> l)`
  reverses the order of the elements in a list. For example, reversing the list `[t, a, r]` yields the list `[r, a, t]`. This method runs in O(n) time, where n is the length of the list.
- `static void rotate(List<?> l, int d)` 1.4
  rotates the elements in the list, moving the entry with index `i` to position `(i + d) % l.size()`. For example, rotating the list `[t, a, r]` by `2` yields the list `[a, r, t]`. This method runs in O(n) time, where n is the length of the list.
- `static int frequency(Collection<?> c, Object o)` 5.0
  returns the count of elements in `c` that equal the object `o`.
- `boolean disjoint(Collection<?> c1, Collection<?> c2)` 5.0
  returns `true` if the collections have no elements in common.

java.util.Collection<T> 1.2

- `default boolean removeIf(Predicate<? super E> filter)` 8
  removes all matching elements.

java.util.List<E> 1.2

- `default void replaceAll(UnaryOperator<E> op)` 8
  applies the operation to all elements of this list.

9.6.5. Bulk Operations

There are several operations that copy or remove elements "in bulk." The call

```
coll1.removeAll(coll2);
```

removes all elements from `coll1` that are present in `coll2`. Conversely,

```
coll1.retainAll(coll2);
```

removes all elements from `coll1` that are *not* present in `coll2`. Here is a typical application.

Suppose you want to find the *intersection* of two sets—the elements that two sets have in common. First, make a new set to hold the result.

```
var result = new HashSet<String>(firstSet);
```

Here, we use the fact that every collection has a constructor whose parameter is another collection that holds the initialization values.

Now, use the `retainAll` method:

```
result.retainAll(secondSet);
```

It retains all elements that occur in both sets. You have formed the intersection without programming a loop.

You can carry this idea further and apply a bulk operation to a *view*. For example, suppose you have a map that maps employee IDs to employee objects, and you have a set of the IDs of all employees that are to be terminated.

```
Map<String, Employee> staffMap = . . .;
Set<String> terminatedIDs = . . .;
```

Simply form the key set and remove all IDs of terminated employees.

```
staffMap.keySet().removeAll(terminatedIDs);
```

Since the key set is a view into the map, the keys and associated employee names are automatically removed from the map.

By using a subrange view, you can restrict bulk operations to sublists and subsets. For example, suppose you want to add the first ten elements of a list to another container. Form a sublist to pick out the first ten:

```
relocated.addAll(staff.subList(0, 10));
```

The subrange can also be a target of a mutating operation.

```
staff.subList(0, 10).clear();
```

9.6.6. Converting between Collections and Arrays

Large portions of the Java API were designed before the Java Collections Framework was created. As a result, you will occasionally need to translate between traditional arrays and the more modern collections.

If you have an array that you need to turn into a collection, the `List.of` method serves this purpose. For example:

```
String[] names = . . .;
List<String> staff = List.of(names);
```

Obtaining an array from a collection is a bit trickier. You can use the `toArray` method:

```
Object[] names = staff.toArray();
```

But the result is an array of *objects*. Even if you know that your collection contained objects of a specific type, you cannot use a cast:

```
String[] names = (String[]) staff.toArray(); // ERROR
```

The array returned by the `toArray` method was created as an `Object[]` array, and you cannot change its type. Instead, pass an array constructor expression to the `toArray` method. The constructor is used to create an array of the correct type:

```
String[] values = staff.toArray(String[]::new);
```



Note: Prior to JDK 11, you had to use another form of the `toArray` method, passing an array of the correct type:

```
String[] values = staff.toArray(new String[0]);
```

This `toArray` method constructs another array of the same type. Or, if the array has sufficient length, it is reused:

```
staff.toArray(new String[staff.size()]);
```

In this case, no new array is created.

9.6.7. Writing Your Own Algorithms

If you write your own algorithm (or, in fact, any method that has a collection as a parameter), you should work with *interfaces*, not concrete implementations, whenever possible. For example, suppose you want to process items. Of course, you can implement a method like this:

```
public void processItems(ArrayList<Item> items) {
    for (Item item : items)
        do something with item
}
```

However, you now constrained the caller of your method—the caller must supply the items in an `ArrayList`. If the items happen to be in another collection, they

first need to be repackaged. It is much better to accept a more general collection.

You should ask yourself this: What is the most general collection interface that can do the job? Do you care about the order? Then you should accept a `List`. But if the order doesn't matter, you can accept collections of any kind:

```
public void processItems(Collection<Item> items) {
    for (Item item : items)
        do something with item
}
```

Now, anyone can call this method with an `ArrayList` or a `LinkedList`, or even with an array wrapped in a call to the `List.of` method.



Tip: In this case, you can do even better by accepting an `Iterable<Item>`. The `Iterable` interface has a single abstract method `iterator` which the enhanced `for` loop uses behind the scenes. The `Collection` interface extends `Iterable`.

Conversely, if your method returns multiple elements, you don't want to constrain yourself against future improvements. For example, consider

```
public ArrayList<Item> lookupItems(. . .) {
    var result = new ArrayList<Item>();
    . . .
    return result;
}
```

This method promises to return an `ArrayList`, even though the caller almost certainly doesn't care what kind of lists it is. If instead you return a `List`, you can at any time add a branch that returns an empty or singleton list by calling `List.of`.



Note: If it is such a good idea to use collection interfaces as parameter and return type, why doesn't the Java API follow this rule consistently? For example, the `JComboBox` class has two constructors:

```
JComboBox(Object[] items)
JComboBox(Vector<?> items)
```

The reason is simply timing. The Swing API was created before the Java Collections Framework.

9.7. Legacy Collections

A number of “legacy” container classes have been present since the first release of Java, before there was a collections framework.

They have been integrated into the Java Collections Framework—see **Figure 9.12**. I will briefly introduce them in the following sections.

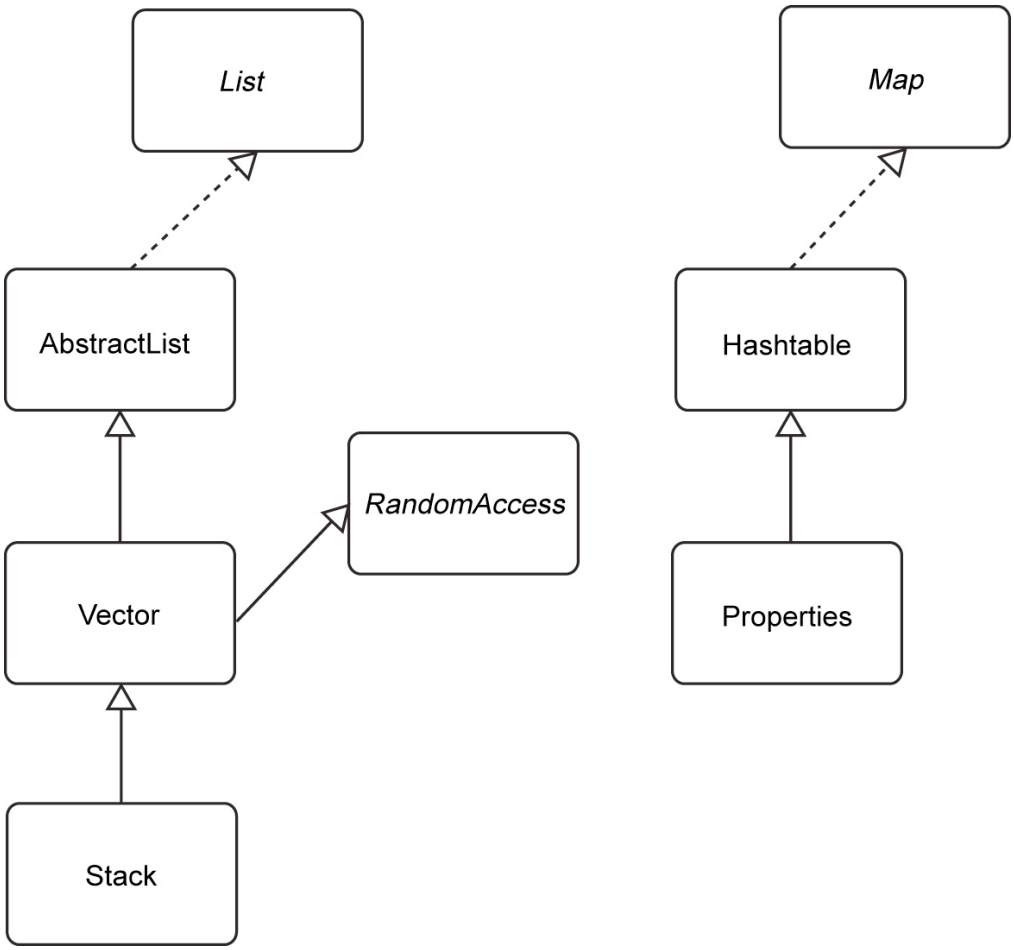


Figure 9.12: Legacy classes in the Java Collections Framework

9.7.1. The `Hashtable` Class

The classic `Hashtable` class serves the same purpose as the `HashMap` class and has essentially the same interface. Just like methods of the `Vector` class, the `Hashtable` methods are synchronized. If you do not require compatibility with legacy code, you should use a `HashMap` instead. If you need concurrent access, use a `ConcurrentHashMap`—see **Chapter 10**.

9.7.2. Enumerations

The legacy collections use the `Enumeration` interface for traversing sequences of elements. The `Enumeration` interface has two methods, `hasMoreElements` and `nextElement`. These are entirely analogous to the `hasNext` and `next` methods of the `Iterator` interface.

If you find this interface with legacy classes, you can use `Collections.list` to collect the elements in an `ArrayList`. For example, the `LogManager` class is only willing to reveal logger names as an `Enumeration`. Here is how you can get them all:

```
ArrayList<String> loggerNames = Collections.list(LogManager.getLoggerNames());
```

Alternatively, you can turn an enumeration into an iterator:

```
LogManager.getLoggerNames().asIterator().forEachRemaining(n -> { . . . });
```

You will occasionally encounter a legacy method with an enumeration parameter. The static method `Collections.enumeration` yields an enumeration object that enumerates the elements in the collection. For example:

```
List<InputStream> streams = . . .;
var in = new SequenceInputStream(Collections.enumeration(streams));
    // the SequenceInputStream constructor expects an enumeration
```



Note: In C++, it is quite common to use iterators as parameters. Fortunately, on the Java platform, very few programmers use this idiom. It is much smarter to pass around the collection than to pass an iterator. The collection object is more useful. The recipients can always obtain the iterator from the collection when they need to do so, plus they have all the collection methods at their disposal. However, you may find enumerations in some legacy code because they were the only available mechanism for generic collections until the Java Collections Framework appeared in Java 1.2.

java.util.Enumeration<E> **1.0**

- `boolean hasMoreElements()`
  returns `true` if there are more elements yet to be inspected.
- `E nextElement()`
  returns the next element to be inspected. Do not call this method if `hasMoreElements()` returned `false`.
- `default Iterator<E> asIterator()` **9**
  yields an iterator that iterates over the enumerated elements.

`java.util.Collections` **1.2**

- `static <T> Enumeration<T> enumeration(Collection<T> c)`
  returns an enumeration that enumerates the elements of `c`.

- `public static <T> ArrayList<T> list(Enumeration<T> e)` returns an array list containing the elements enumerated by `e`.

9.7.3. Property Maps

A *property map* is a map structure of a special type. It has three particular characteristics:

- The keys and values are strings.
- The map can easily be saved to a file and loaded from a file.
- There is a secondary table for default values.

The Java platform class that implements a property map is called `Properties`. Property maps are useful in specifying configuration options for programs. For example:

```
var settings = new Properties();
settings.setProperty("width", "600.0");
settings.setProperty("filename", "/home/cay/books/corejava/code/v1ch09/raven.html");
```

Use the `getProperty` method to look up the value for a key:

```
String filename = settings.getProperty("filename");
```



Caution: For historical reasons, the `Properties` class implements `Map<Object, Object>`. Therefore, you can use the `get` and `put` methods of the `Map` interface. But the `get` method returns the type `Object`, and the `put` method allows you to insert any object. It is best to stick with the `getProperty` and `setProperty` methods that work with strings, not objects.

Use the `store` method to save the properties to a file. The second argument is a comment that is included in the file.

```
var out = new FileWriter("program.properties");
settings.store(out, "Program Properties");
```

The sample set gives the following output:

```
#Program Properties
#Sun Dec 31 12:54:19 PST 2025
top=227.0
left=1286.0
width=423.0
height=547.0
filename=/home/cay/books/corejava/code/v1ch09/raven.html
```

To load the properties from a file, use

```
var in = new FileReader("program.properties");
settings.load(in);
```



Caution: If you use the `load` and `store` methods with input/output streams, then the archaic ISO 8859-1 character encoding is used, and characters > U+00FF are saved as Unicode escapes. For UTF-8, use readers/writers, as in the code snippets above. Prior to Java 18, set the character encoding explicitly to `StandardCharsets.UTF_8`.

The `Properties` class has two mechanisms for providing defaults. First, whenever you look up the value of a string, you can specify a default that should be used automatically when the key is not present.

```
String filename = settings.getProperty("filename", "");
```

If there is a `"filename"` property in the property map, `filename` is set to that string. Otherwise, `filename` is set to the empty string.

If you find it too tedious to specify the default in every call to `getProperty`, you can pack all the defaults into a secondary property map and supply that map in the constructor of your primary property map.

```
var defaultSettings = new Properties();
defaultSettings.setProperty("width", "600");
defaultSettings.setProperty("height", "400");
defaultSettings.setProperty("filename", "");
. . .
var settings = new Properties(defaultSettings);
```

Yes, you can even specify defaults to defaults if you give another property map argument to the `defaultSettings` constructor, but it is not something one would normally do.

The companion code has a sample program that shows how you can use properties for storing and loading program state. The program uses the `ImageViewer` program from **Chapter 2** and remembers the frame position, size, and last loaded file. Run the program, load a file, and move and resize the window. Then close the program and reopen it to see that it remembers your file and your favorite window placement. You can also manually edit the file `.corejava/ImageViewer.properties` in your home directory.

Properties are simple tables without a hierarchical structure. It is common to introduce a fake hierarchy with key names such as `window.main.color`, `window.main.title`, and so on. But the `Properties` class has no methods that

help organize such a hierarchy. If you store complex configuration information, you should use the `Preferences` class instead—see **Chapter 10 of Volume II**.

`java.util.Properties` 1.0

- `Properties()`
  creates an empty property map.
- `Properties(Properties defaults)`
  creates an empty property map with a map of defaults.
- `String getProperty(String key)`
  gets a property. Returns the string associated with the key, or the string associated with the key in the default table if it wasn't present in the table, or `null` if the key wasn't present in the default table either.
- `String getProperty(String key, String defaultValue)`
  gets a property with a default value if the key is not found. Returns the string associated with the key, or the default string if it wasn't present in the table.
- `Object setProperty(String key, String value)`
  sets a property. Returns the previously set value of the given key.
- `Set<String> stringPropertyNames()` 6
  returns a set of all keys, including the keys from the default map.
- `void load(Reader in) throws IOException` 6
  loads a property map from a reader.
- `void store(Writer out, String header)` 6
  saves a property map to a writer. The header is in the first line of the stored file.

9.7.4. System Properties

The `System.getProperties` method yields a `Properties` object to describe system information. For example, the home directory has the key `"user.home"`.

You can read it with the `getProperty` method that yields the key as a string:

```
String userDir = System.getProperty("user.home");
```

To get the Java version of the virtual machine, look up the `"java.version"` property. You get a string such as `"21.0.1"` (but `"1.8.0"` up to Java 8).



Tip: As you can see, the version numbering changed in Java 9. This seemingly small change broke a good number of tools that had relied on the old format. If you parse the version string, be sure to read JEP 322 at **https://openjdk.org/jeps/322** to see how version strings will be formatted in the future—or at least, until the numbering scheme changes again.



Tip: To print out all system properties, run:

```
java -XshowSettings:properties
```

The `java.version` system property is set by the virtual machine and should not be changed. Other system properties can be set with the `-D` command line option:

```
java -Duser.language=fr -Duser.country=CA MyProgram
```

To make properties available in `jshell`, use the `-R` option like this:

```
jshell -R-Duser.language=fr -R-Duser.country=CA
```

To get a numeric or Boolean system property value, use the static methods `Boolean.getBoolean()`, `Integer.getInteger`, and `Long.getLong`:

```
Integer version = Integer.getInteger("java.specification.version");
```

`java.lang.System` **1.0**

- `Properties getProperties()`
  retrieves all system properties. The application must have permission to retrieve all properties, or a security exception is thrown.
- `String getProperty(String key)`
  retrieves the system property with the given key name. The following properties are always present:

  ```
  java.version
  java.version.date
  java.vendor
  java.vendor.url
  java.home
  java.class.path
  java.library.path
  java.class.version
  java.specification.version
  java.specification.vendor
  java.specification.name
  java.vm.specification.version
  java.vm.specification.vendor
  java.vm.specification.name
  java.vm.version
  java.vm.vendor
  java.vm.name
  os.name
  os.version
  os.arch
  ```

```
file.separator
path.separator
line.separator
java.io.tmpdir
user.name
user.home
user.dir
native.encoding
stdout.encoding
stderr.encoding
```

`java.lang.Boolean` 1.0

- `static boolean getBoolean(String name)`
  returns `true` if the system property with the given name has value `"true"` (ignoring letter case), `false` otherwise.

`java.lang.Integer` 1.0

- `static Integer getInteger(String nm)`
- `static Integer getInteger(String nm, int val)`
- `static Integer getInteger(String nm, Integer val)`
  returns the result of calling `Integer.decode` on the system property with the given name. If there is no system property with the given name, or its value cannot be decoded, this method returns the default value or, if none provided, `null`.

`java.lang.Long` 1.0

- `static Long getLong(String nm)`
- `static Long getLong(String nm, int val)`
- `static Long getLong(String nm, Long val)`
  returns the result of calling `Long.decode` on the system property with the given name. If there is no system property with the given name, or its value cannot be decoded, this method returns the default value or, if none provided, `null`.

9.7.5. Stacks

Since version 1.0, the Java API had a `Stack` class with the familiar `push` and `pop` methods. However, the `Stack` class extends the `Vector` class, which is not satisfactory from a theoretical perspective—you can apply such un-stack-like operations as `insert` and `remove` to insert and remove values anywhere, not just at the top of the stack.

`java.util.Stack<E>` 1.0

- `E push(E item)`
  pushes `item` onto the stack and returns `item`.
- `E pop()`
  pops and returns the top item of the stack. Don't call this method if the stack is empty.
- `E peek()`
  returns the top of the stack without popping it. Don't call this method if the stack is empty.

9.7.6. Bit Sets

The Java platform's `BitSet` class stores a sequence of bits, packed into implementation-specific "words", which are currently `long` values. That is much more efficient than using an array of `boolean` values.

The `BitSet` class gives you a convenient interface for reading, setting, and resetting individual bits. Using this interface avoids the masking and other bit-fiddling operations that are necessary if you store bits in `int` or `long` variables.

For example, for a `BitSet` named `bucketOfBits`,

```
bucketOfBits.get(i)
```

returns `true` if the `i` th bit is on, and `false` otherwise. Similarly,

```
bucketOfBits.set(i)
```

turns the `i` th bit on. Finally,

```
bucketOfBits.clear(i)
```

turns the `i` th bit off.

You can use a `BitSet` to represent a set of nonnegative integers with an upper bound. Set the `i` th bit to indicate that the integer `i` is in the set. This is more efficient than using a `Set<Integer>` if the upper bound is not too large and the set has many elements.

`java.util.BitSet` 1.0

- `BitSet(int initialCapacity)`
  constructs a bit set.

- `int cardinality()` 1.4
  returns the number of bits that are set, or, when considered as a set of integers, the number of elements.
- `int length()` 1.2
  returns the "logical length" (1 plus the index of the highest set bit). This is useful for iterating over the elements.
- `int size()`
  returns the number of bits currently available in the internal data structure, *not* the number of set elements.
- `boolean get(int bit)`
  gets a bit.
- `void set(int bit)`
  sets a bit.
- `void clear(int bit)`
  clears a bit.
- `void and(BitSet set)`
  logically ANDs this bit set with another.
- `void or(BitSet set)`
  logically ORs this bit set with another.
- `void xor(BitSet set)`
  logically XORs this bit set with another.
- `void andNot(BitSet set)`
  clears all bits in this bit set that are set in the other bit set.
- `IntStream stream()` 8
  yields a stream of the index values of the bits that are set, or, when considered as a set of integers, a stream of the elements.

As an example of using bit sets, I want to show you an implementation of the "sieve of Eratosthenes" algorithm for finding prime numbers. (A prime number is a number like 2, 3, or 5 that is divisible only by itself and 1, and the sieve of Eratosthenes was one of the first methods discovered to enumerate these fundamental building blocks.) This isn't a terribly good algorithm for finding the primes, but for some reason it has become a popular benchmark for compiler performance. (It isn't a good benchmark either, because it mainly tests bit operations.)

Oh well, I bow to tradition and present an implementation. This program counts all prime numbers between 2 and 2,000,000. (There are 148,933 primes in this interval, so you probably don't want to print them all out.)

Without going into too many details of this program, the idea is to march through a bit set with 2 million bits. First, we turn on all the bits. After that, we turn off the bits that are multiples of numbers known to be prime. The positions of the bits that remain after this process are themselves prime numbers. **Listing 9.7** lists this program in the Java programming language, and **Listing 9.8** is the C++ code.



Note: Even though the sieve isn't a good benchmark, I couldn't resist timing the two implementations of the algorithm. Here are the timing results with an Intel i7-1165G7 processor and 32 GB of RAM, running Ubuntu 24.04:

- C++ (g++ 11.4.0): 70 milliseconds
- Java (Java 25): 20 milliseconds

I have run this test for fourteen editions of *Core Java*, and in the last ten editions, Java easily beat C++. In all fairness, if one cranks up the optimization level in the C++ compiler, it beats Java with a time of 16 milliseconds. Java could only match that if the program ran long enough to trigger the Hotspot just-in-time compiler.

Listing 9.7 v1ch09/sieve/Sieve.java

```
1  package v1ch09.sieve;
2
3  import module java.base;
4
5  /**
6   * This program runs the Sieve of Erathostenes benchmark. It computes all primes
7   * up to 2,000,000.
8   */
9  public class Sieve {
10     void main() {
11         int n = 2000000;
12         long start = System.nanoTime();
13         var bitSet = new BitSet(n + 1);
14         int i;
15         for (i = 2; i <= n; i++)
16             bitSet.set(i);
17         i = 2;
18         while (i * i <= n) {
19             if (bitSet.get(i)) {
20                 int k = i * i;
21                 while (k <= n) {
22                     bitSet.clear(k);
23                     k += i;
24                 }
25             }
26             i++;
27         }
28         long end = System.nanoTime();
29         IO.println(bitSet.cardinality() + " primes");
30         IO.println((end - start) / 1000 + " milliseconds");
31     }
32 }
```

Listing 9.8 v1ch09/sieve/sieve.cpp

```
#include <bitset>
#include <iostream>
#include <ctime>

using namespace std;

int main() {
    const int N = 2000000;
    clock_t cstart = clock();

    bitset<N + 1> b;
    int i;
    for (i = 2; i <= N; i++)
        b.set(i);
    i = 2;
    while (i * i <= N) {
        if (b.test(i)) {
            int k = i * i;
            while (k <= N) {
                b.reset(k);
                k += i;
            }
        }
       i++;
    }

    clock_t cend = clock();
    double millis = 1000.0 * (cend - cstart) / CLOCKS_PER_SEC;

    cout << b.count() << " primes\n" << millis << " milliseconds\n";

    return 0;
}
```

This completes our tour through the Java Collections Framework. As you have seen, the framework offers a wide variety of collection classes for your programming needs.

Previous chapter

< 8. Generic Programming

Next chapter

10. Concurrency >